

Chapter 12. The Virtual Filesystem

One of Linux's keys to success is its ability to coexist comfortably with other systems. You can transparently mount disks or partitions that host file formats used by Windows , other Unix systems, or even systems with tiny market shares like the Amiga. Linux manages to support multiple filesystem types in the same way other Unix variants do, through a concept called the Virtual Filesystem.

The idea behind the Virtual Filesystem is to put a wide range of information in the kernel to represent many different types of filesystems ; there is a field or function to support each operation provided by all real filesystems supported by Linux. For each read, write, or other function called, the kernel substitutes the actual function that supports a native Linux filesystem, the NTFS filesystem, or whatever other filesystem the file is on.

This chapter discusses the aims, structure, and implementation of Linux's Virtual Filesystem. It focuses on three of the five standard Unix file typesnamely, regular files, directories, and symbolic links. Device files are covered in [Chapter 13](#), while pipes are discussed in [Chapter 19](#). To show how a real filesystem works, [Chapter 18](#) covers the Second Extended Filesystem that appears on nearly all Linux systems.

12.1. The Role of the Virtual Filesystem (VFS)

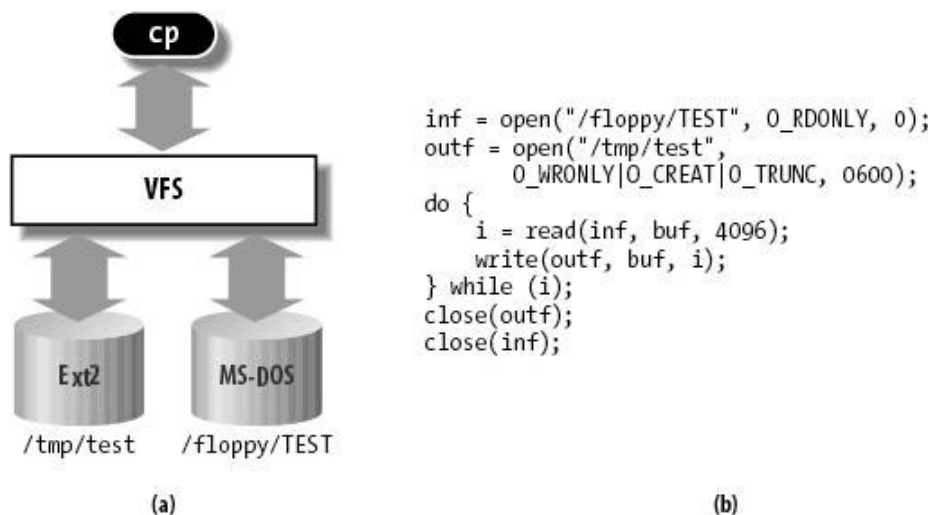
The Virtual Filesystem (also known as Virtual Filesystem Switch or VFS) is a kernel software layer that handles all system calls related to a standard Unix filesystem. Its main strength is providing a common interface to several kinds of filesystems.

For instance, let's assume that a user issues the shell command:

```
$ cp /floppy/TEST /tmp/test
```

where /floppy is the mount point of an MS-DOS diskette and /tmp is a normal Second Extended Filesystem (Ext2) directory. The VFS is an abstraction layer between the application program and the filesystem implementations (see [Figure 12-1\(a\)](#)). Therefore, the cp program is not required to know the filesystem types of /floppy/TEST and /tmp/test. Instead, cp interacts with the VFS by means of generic system calls known to anyone who has done Unix programming (see the section "[File-Handling System Calls](#)" in [Chapter 1](#)); the code executed by cp is shown in [Figure 12-1\(b\)](#).

Figure 12-1. VFS role in a simple file copy operation



Filesystems supported by the VFS may be grouped into three main classes:

Disk-based filesystems

These manage memory space available in a local disk or in some other device that emulates a disk (such as a USB flash drive). Some of the well-known disk-based filesystems supported by the VFS are:

- ◊ Filesystems for Linux such as the widely used Second Extended Filesystem (Ext2), the recent Third Extended Filesystem (Ext3), and the Reiser Filesystems (ReiserFS)^[*]

^[*] Although these filesystems owe their birth to Linux, they have been ported to several other operating systems.

- ◊ Filesystems for Unix variants such as sysv filesystem (System V , Coherent , Xenix), UFS (BSD , Solaris , NEXTSTEP), MINIX filesystem, and VERITAS VxFS (SCO UnixWare)
- ◊ Microsoft filesystems such as MS-DOS, VFAT (Windows 95 and later releases), and NTFS (Windows NT 4 and later releases)
- ◊ ISO9660 CD-ROM filesystem (formerly High Sierra Filesystem) and Universal Disk Format (UDF) DVD filesystem
- ◊ Other proprietary filesystems such as IBM's OS/2 (HPFS), Apple's Macintosh (HFS), Amiga's Fast Filesystem (AFFS), and Acorn Disk Filing System (ADFS)
- ◊ Additional journaling filesystems originating in systems other than Linux such as IBM's JFS and SGI's XFS

Network filesystems

These allow easy access to files included in filesystems belonging to other networked computers. Some well-known network filesystems supported by the VFS are NFS , Coda , AFS (Andrew filesystem), CIFS (Common Internet File System, used in Microsoft Windows), and NCP (Novell's NetWare Core Protocol).

Special filesystems

These do not manage disk space, either locally or remotely. The */proc* filesystem is a typical example of a special filesystem (see the later section "[Special Filesystems](#)").

In this book, we describe in detail the Ext2 and Ext3 filesystems only (see [Chapter 18](#)); the other filesystems are not covered for lack of space.

As mentioned in the section "[An Overview of the Unix Filesystem](#)" in [Chapter 1](#), Unix directories build a tree whose root is the */* directory. The root directory is contained in the root filesystem, which in Linux, is usually of type Ext2 or Ext3. All other filesystems can be "mounted" on subdirectories of the root filesystem.^[*]

^[*] When a filesystem is mounted on a directory, the contents of the directory in the parent filesystem are no longer accessible, because every pathname, including the mount point, will refer to the mounted filesystem. However, the original directory's content shows up again when the filesystem is unmounted. This somewhat surprising feature of Unix filesystems is used by system administrators to hide files; they simply mount a filesystem on the directory containing the files to be hidden.

A disk-based filesystem is usually stored in a hardware block device such as a hard disk, a floppy, or a CD-ROM. A useful feature of Linux's VFS allows it to handle virtual block devices such as */dev/loop0*, which may be used to mount filesystems stored in regular files. As a possible application, a user may protect her own private filesystem by storing an encrypted version of it in a regular file.

The first Virtual Filesystem was included in Sun Microsystems's SunOS in 1986. Since then,

most Unix filesystems include a VFS. Linux's VFS, however, supports the widest range of filesystems.

12.1.1. The Common File Model

The key idea behind the VFS consists of introducing a common file model capable of representing all supported filesystems. This model strictly mirrors the file model provided by the traditional Unix filesystem. This is not surprising, because Linux wants to run its native filesystem with minimum overhead. However, each specific filesystem implementation must translate its physical organization into the VFS's common file model.

For instance, in the common file model, each directory is regarded as a file, which contains a list of files and other directories. However, several non-Unix disk-based filesystems use a File Allocation Table (FAT), which stores the position of each file in the directory tree. In these filesystems, directories are not files. To stick to the VFS's common file model, the Linux implementations of such FAT-based filesystems must be able to construct on the fly, when needed, the files corresponding to the directories. Such files exist only as objects in kernel memory.

More essentially, the Linux kernel cannot hardcode a particular function to handle an operation such as `read( )` or `ioctl( )`. Instead, it must use a pointer for each operation; the pointer is made to point to the proper function for the particular filesystem being accessed.

Let's illustrate this concept by showing how the `read( )` shown in [Figure 12-1](#) would be translated by the kernel into a call specific to the MS-DOS filesystem. The application's call to `read( )` makes the kernel invoke the corresponding `sys_read( )` service routine, like every other system call. The file is represented by a `file` data structure in kernel memory, as we'll see later in this chapter. This data structure contains a field called `f_op` that contains pointers to functions specific to MS-DOS files, including a function that reads a file. `sys_read( )` finds the pointer to this function and invokes it. Thus, the application's `read( )` is turned into the rather indirect call:

```
file->f_op->read(...);
```

Similarly, the `write( )` operation triggers the execution of a proper Ext2 write function associated with the output file. In short, the kernel is responsible for assigning the right set of pointers to the `file` variable associated with each open file, and then for invoking the call specific to each filesystem that the `f_op` field points to.

One can think of the common file model as object-oriented, where an object is a software construct that defines both a data structure and the methods that operate on it. For reasons of efficiency, Linux is not coded in an object-oriented language such as C++. Objects are therefore implemented as plain C data structures with some fields pointing to functions that correspond to the object's methods.

The common file model consists of the following object types:

The superblock object

Stores information concerning a mounted filesystem. For disk-based filesystems, this object usually corresponds to a filesystem control block stored on disk.

The inode object

Stores general information about a specific file. For disk-based filesystems, this object usually corresponds to a file control block stored on disk. Each inode object is associated with an inode number, which uniquely identifies the file within the filesystem.

The file object

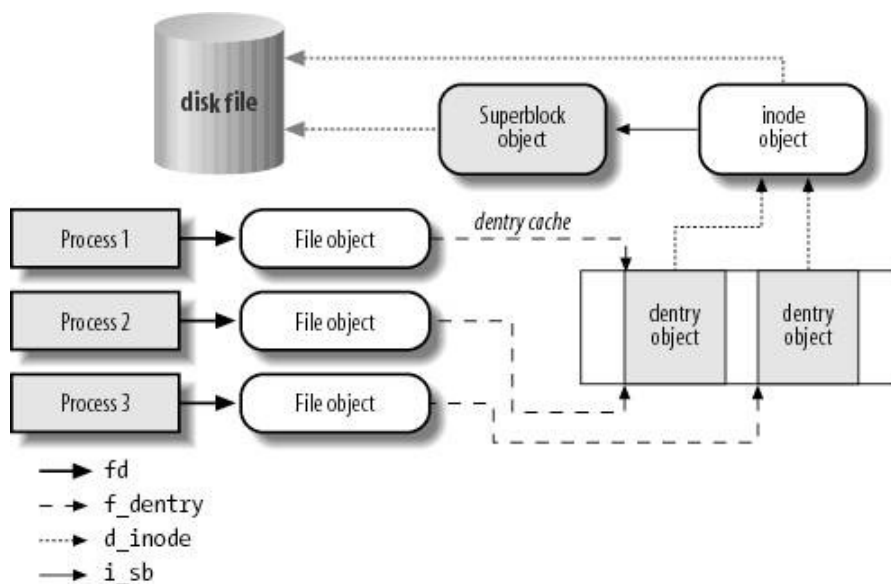
Stores information about the interaction between an open file and a process. This information exists only in kernel memory during the period when a process has the file open.

The dentry object

Stores information about the linking of a directory entry (that is, a particular name of the file) with the corresponding file. Each disk-based filesystem stores this information in its own particular way on disk.

Figure 12-2 illustrates with a simple example how processes interact with files. Three different processes have opened the same file, two of them using the same hard link. In this case, each of the three processes uses its own file object, while only two dentry objects are required for each hard link. Both dentry objects refer to the same inode object, which identifies the superblock object and, together with the latter, the common disk file.

Figure 12-2. Interaction between processes and VFS objects



Besides providing a common interface to all filesystem implementations, the VFS has another important role related to system performance. The most recently used dentry objects

are contained in a disk cache named the dentry cache , which speeds up the translation from a file pathname to the inode of the last pathname component.

Generally speaking, a disk cache is a software mechanism that allows the kernel to keep in RAM some information that is normally stored on a disk, so that further accesses to that data can be quickly satisfied without a slow access to the disk itself.

Notice how a disk cache differs from a hardware cache or a memory cache, neither of which has anything to do with disks or other devices. A hardware cache is a fast static RAM that speeds up requests directed to the slower dynamic RAM (see the section "[Hardware Cache](#)" in [Chapter 2](#)). A memory cache is a software mechanism introduced to bypass the Kernel Memory Allocator (see the section "[The Slab Allocator](#)" in [Chapter 8](#)).

Beside the dentry cache and the inode cache, Linux uses other disk caches. The most important one, called the page cache, is described in detail in [Chapter 15](#).

12.1.2. System Calls Handled by the VFS

[Table 12-1](#) illustrates the VFS system calls that refer to filesystems, regular files, directories, and symbolic links. A few other system calls handled by the VFS, such as `ioperm( )` , `ioctl( )` , `pipe( )` , and `mknod( )` , refer to device files and pipes. These are discussed in later chapters. A last group of system calls handled by the VFS, such as `socket( )` , `connect( )` , and `bind( )` , refer to sockets and are used to implement networking. Some of the kernel service routines that correspond to the system calls listed in [Table 12-1](#) are discussed either in this chapter or in [Chapter 18](#).

Table 12-1. Some system calls handled by the VFS

System call name	Description
<code>mount()</code> <code>umount()</code> <code>umount2()</code>	Mount/unmount filesystems
<code>sysfs()</code>	Get filesystem information
<code>statfs()</code> <code>fstatfs()</code> <code>statfs64()</code> <code>fstatfs64()</code>	Get filesystem statistics
<code>ustat()</code>	
<code>chroot()</code> <code>pivot_root()</code>	Change root directory
<code>chdir()</code> <code>fchdir()</code> <code>getcwd()</code>	Manipulate current directory
<code>mkdir()</code> <code>rmdir()</code>	Create and destroy directories
<code>getdents()</code> <code>getdents64()</code> <code>readdir()</code> <code>link()</code>	Manipulate directory entries
<code>unlink()</code> <code>rename()</code> <code>lookup_dcookie()</code>	
<code>readlink()</code> <code>symlink()</code>	Manipulate soft links
<code>chown()</code> <code>fchown()</code> <code>lchown()</code> <code>chown16()</code>	Modify file owner
<code>fchown16()</code> <code>lchown16()</code>	
<code>chmod()</code> <code>fchmod()</code> <code>utime()</code>	

	Modify file attributes
<code>stat() fstat() lstat() access() oldstat() oldfstat() oldlstat() stat64() lstat64()</code>	Read file status
<code>fstat64()</code>	
<code>open() close() creat() umask()</code>	Open, close, and create files
<code>dup() dup2() fcntl() fcntl64()</code>	Manipulate file descriptors
<code>select() poll()</code>	Wait for events on a set of file descriptors
<code>truncate() ftruncate() truncate64()</code>	Change file size
<code>ftruncate64()</code>	
<code>lseek() _llseek()</code>	Change file pointer
<code>read() write() readv() writev() sendfile() sendfile64()</code>	Carry out file I/O operations
<code>readahead()</code>	Asynchronous I/O (allows multiple outstanding read and write requests)
<code>io_setup() io_submit() io_getevents() io_cancel() io_destroy()</code>	Seek file and access it
<code>pread64() pwrite64()</code>	Handle file memory mapping
<code>mmap() mmap2() munmap() madvise() mincore()</code>	Synchronize file data
<code>remap_file_pages()</code>	Manipulate file lock
<code>fdatasync() fsync() sync() msync()</code>	Manipulate file extended attributes
<code>flock()</code>	
<code>setxattr() lsetxattr() fsetxattr() getxattr() lgetxattr() fgetxattr() listxattr() llistxattr() flistxattr() removexattr() lremovexattr() fremovexattr()</code>	

We said earlier that the VFS is a layer between application programs and specific filesystems. However, in some cases, a file operation can be performed by the VFS itself, without invoking a lower-level procedure. For instance, when a process closes an open file, the file on disk doesn't usually need to be touched, and hence the VFS simply releases the corresponding file object. Similarly, when the `lseek( )` system call modifies a file pointer, which is an attribute related to the interaction between an opened file and a process, the VFS needs to modify only the corresponding file object without accessing the file on disk, and therefore it does not have to invoke a specific filesystem procedure. In some sense, the VFS could be considered a "generic" filesystem that relies, when necessary, on specific ones.

12.2. VFS Data Structures

Each VFS object is stored in a suitable data structure, which includes both the object attributes and a pointer to a table of object methods. The kernel may dynamically modify the methods of the object and, hence, it may install specialized behavior for the object. The following sections explain the VFS objects and their interrelationships in detail.

12.2.1. Superblock Objects

A superblock object consists of a `super_block` structure whose fields are described in [Table 12-2](#).

Table 12-2. The fields of the superblock object

Type	Field	Description
<code>struct list_head</code>	<code>s_list</code>	Pointers for superblock list
<code>dev_t</code>	<code>s_dev</code>	Device identifier
<code>unsigned long</code>	<code>s_blocksize</code>	Block size in bytes
<code>unsigned long</code>	<code>s_old_blocksize</code>	Block size in bytes as reported by the underlying block device driver
<code>unsigned char</code>	<code>s_blocksize_bits</code>	Block size in number of bits
<code>unsigned char</code>	<code>s_dirt</code>	Modified (dirty) flag
<code>unsigned long long</code>	<code>s_maxbytes</code>	Maximum size of the files
<code>struct</code>	<code>s_type</code>	Filesystem type
<code>file_system_type *</code>	<code>s_op</code>	Superblock methods
<code>struct</code>	<code>dq_op</code>	Disk quota handling methods
<code>super_operations *</code>	<code>s_qcop</code>	Disk quota administration methods
<code>struct dqquot_operations *</code>	<code>s_export_op</code>	Export operations used by network filesystems
<code>struct quotactl_ops *</code>	<code>s_flags</code>	Mount flags
<code>struct export_operations *</code>	<code>s_magic</code>	Filesystem magic number
<code>unsigned long</code>	<code>s_root</code>	Dentry object of the filesystem's root directory
<code>unsigned long</code>	<code>s_umount</code>	Semaphore used for
<code>struct dentry *</code>		
<code>struct rw_semaphore</code>		

		unmounting
struct semaphore	s_lock	Superblock semaphore
int	s_count	Reference counter
		Flag indicating that
int	s_syncing	inodes of the superblock
		are being synchronized
		Flag used when
int	s_need_sync_fs	synchronizing the
		superblock's mounted
		filesystem
atomic_t	s_active	Secondary reference
		counter
void *	s_security	Pointer to superblock
		security structure
struct xattr_handler **	s_xattr	Pointer to superblock
		extended attribute
		structure
struct list_head	s_inodes	List of all inodes
struct list_head	s_dirty	List of modified inodes
struct list_head	s_io	List of inodes waiting to
		be written to disk
		List of anonymous
struct hlist_head	s_anon	dentries for handling
		remote network
		filesystems
struct list_head	s_files	List of file objects
struct block_device *	s_bdev	Pointer to the block
		device driver descriptor
		Pointers for a list of
		superblock objects of a
struct list_head	s_instances	given filesystem type (see
		the later section
		"Filesystem Type
		Registration")
struct quota_info	s_dquot	Descriptor for disk quota
		Flag used when freezing
int	s_frozen	the filesystem (forcing it
		to a consistent state)
		Wait queue where
wait_queue_head_t	s_wait_unfrozen	processes sleep until the
		filesystem is unfrozen
char[]	s_id	Name of the block device
		containing the superblock
		Pointer to superblock
void *	s_fs_info	information of a specific
		filesystem
		Semaphore used by VFS
struct semaphore	s_vfs_rename_sem	when renaming files
		across directories
u32	s_time_gran	Timestamp's granularity
		(in nanoseconds)

All superblock objects are linked in a circular doubly linked list. The first element of this list is represented by

the `super_blocks` variable, while the `s_list` field of the superblock object stores the pointers to the adjacent elements in the list. The `sb_lock` spin lock protects the list against concurrent accesses in multiprocessor systems.

The `s_fs_info` field points to superblock information that belongs to a specific filesystem; for instance, as we'll see later in [Chapter 18](#), if the superblock object refers to an Ext2 filesystem, the field points to an `ext2_sb_info` structure, which includes the disk allocation bit masks and other data of no concern to the VFS common file model.

In general, data pointed to by the `s_fs_info` field is information from the disk duplicated in memory for reasons of efficiency. Each disk-based filesystem needs to access and update its allocation bitmaps in order to allocate or release disk blocks. The VFS allows these filesystems to act directly on the `s_fs_info` field of the superblock in memory without accessing the disk.

This approach leads to a new problem, however: the VFS superblock might end up no longer synchronized with the corresponding superblock on disk. It is thus necessary to introduce an `s_dirt` flag, which specifies whether the superblock is dirty—that is, whether the data on the disk must be updated. The lack of synchronization leads to the familiar problem of a corrupted filesystem when a site's power goes down without giving the user the chance to shut down a system cleanly. As we'll see in the section "[Writing Dirty Pages to Disk](#)" in [Chapter 15](#), Linux minimizes this problem by periodically copying all dirty superblocks to disk.

The methods associated with a superblock are called superblock operations. They are described by the `super_operations` structure whose address is included in the `s_op` field.

Each specific filesystem can define its own superblock operations. When the VFS needs to invoke one of them, say `read_inode()`, it executes the following:

```
sb->s_op->read_inode(inode);
```

where `sb` stores the address of the superblock object involved. The `read_inode` field of the `super_operations` table contains the address of the suitable function, which is therefore directly invoked.

Let's briefly describe the superblock operations, which implement higher-level operations like deleting files or mounting disks. They are listed in the order they appear in the `super_operations` table:

`alloc_inode(sb)`

Allocates space for an inode object, including the space required for filesystem-specific data.

`destroy_inode(inode)`

Destroys an inode object, including the filesystem-specific data.

`read_inode(inode)`

Fills the fields of the inode object passed as the parameter with the data on disk; the `i_ino` field of the inode object identifies the specific filesystem inode on the disk to be read.

`dirty_inode(inode)`

Invoked when the inode is marked as modified (dirty). Used by filesystems such as ReiserFS and Ext3 to update the filesystem journal on disk.

`write_inode(inode, flag)`

Updates a filesystem inode with the contents of the inode object passed as the parameter; the `i_ino` field of the inode object identifies the filesystem inode on disk that is concerned. The `flag` parameter indicates whether the I/O operation should be synchronous.

`put_inode(inode)`

Invoked when the inode is released its reference counter is decreased to perform filesystem-specific operations.

`drop_inode(inode)`

Invoked when the inode is about to be destroyed that is, when the last user releases the inode; filesystems that implement this method usually make use of `generic_drop_inode()`. This function removes every reference to the inode from the VFS data structures and, if the inode no longer appears in any directory, invokes the `delete_inode` superblock method to delete the inode from the filesystem.

`delete_inode(inode)`

Invoked when the inode must be destroyed. Deletes the VFS inode in memory and the file data and metadata on disk.

`put_super(super)`

Releases the superblock object passed as the parameter (because the corresponding filesystem is unmounted).

`write_super(super)`

Updates a filesystem superblock with the contents of the object indicated.

`sync_fs(sb, wait)`

Invoked when flushing the filesystem to update filesystem-specific data structures on disk (used by journaling filesystems).

`write_super_lockfs(super)`

Blocks changes to the filesystem and updates the superblock with the contents of the object indicated. This method is invoked when the filesystem is frozen, for instance by the Logical Volume Manager (LVM) driver.

`unlockfs(super)`

Undoes the block of filesystem updates achieved by the `write_super_lockfs` superblock method.

`statfs(super, buf)`

Returns statistics on a filesystem by filling the `buf` buffer.

`remount_fs(super, flags, data)`

Remounts the filesystem with new options (invoked when a mount option must be changed).

`clear_inode(inode)`

Invoked when a disk inode is being destroyed to perform filesystem-specific operations.

`umount_begin(super)`

Aborts a mount operation because the corresponding unmount operation has been started (used only by network filesystems).

`show_options(seq_file, vfsmount)`

Used to display the filesystem-specific options

`quota_read(super, type, data, size, offset)`

Used by the quota system to read data from the file that specifies the limits for this filesystem.^[*]

^[*] The quota system defines for each user and/or group limits on the amount of space that can be used on a given filesystem (see the `quotactl()` system call.)

`quota_write(super, type, data, size, offset)`

Used by the quota system to write data into the file that specifies the limits for this filesystem.

The preceding methods are available to all possible filesystem types. However, only a subset of them applies to each specific filesystem; the fields corresponding to unimplemented methods are set to `NULL`. Notice that no `get_super` method to read a superblock is defined how could the kernel invoke a method of an object yet to be read from disk? We'll find an equivalent `get_sb` method in another object describing the filesystem type (see the later section "[Filesystem Type Registration](#)").

12.2.2. Inode Objects

All information needed by the filesystem to handle a file is included in a data structure called an inode. A filename is a casually assigned label that can be changed, but the inode is unique to the file and remains the same as long as the file exists. An inode object in memory consists of an `inode` structure whose fields are described in [Table 12-3](#).

Table 12-3. The fields of the inode object

Type

Field

Description

6

struct hlist_node

i_hash

Pointers for the hash list

struct list_head

i_list

Pointers for the list that describes the inode's current state

struct list_head

i_sb_list

Pointers for the list of inodes of the superblock

struct list_head

i_dentry

The head of the list of dentry objects referencing this inode

unsigned long

i_ino

inode number

atomic_t

i_count

Usage counter

umode_t

i_mode

File type and access rights

unsigned int

i_nlink

Number of hard links

uid_t

i_uid

Owner identifier

6

`gid_t`

`i_gid`

Group identifier

`dev_t`

`i_rdev`

Real device identifier

`loff_t`

`i_size`

File length in bytes

`struct timespec`

`i_atime`

Time of last file access

`struct timespec`

`i_mtime`

Time of last file write

`struct timespec`

`i_ctime`

Time of last inode change

`unsigned int`

`i_blkbits`

Block size in number of bits

`unsigned long`

`i_blksize`

Block size in bytes

`unsigned long`

`i_version`

Version number, automatically increased after each use

8

`unsigned long`

`i_blocks`

Number of blocks of the file

`unsigned short`

`i_bytes`

Number of bytes in the last block of the file

`unsigned char`

`i_sock`

Nonzero if file is a socket

`spinlock_t`

`i_lock`

Spin lock protecting some fields of the inode

`struct semaphore`

`i_sem`

inode semaphore

`struct rw_semaphore`

`i_alloc_sem`

Read/write semaphore protecting against race conditions in direct I/O file operations

`struct inode_operations *`

`i_op`

inode operations

`struct file_operations *`

`i_fop`

Default file operations

`struct super_block *`

`i_sb`

Pointer to superblock object

8

```
struct file_lock *
```

```
i_flock
```

Pointer to file lock list

```
struct address_space *
```

```
i_mapping
```

Pointer to an `address_space` object (see [Chapter 15](#))

```
struct address_space
```

```
i_data
```

`address_space` object of the file

```
struct dquot * []
```

```
i_dquot
```

inode disk quotas

```
struct list_head
```

```
i_devices
```

Pointers for a list of inodes relative to a specific character or block device (see [Chapter 13](#))

```
struct pipe_inode_info *
```

```
i_pipe
```

Used if the file is a pipe (see [Chapter 19](#))

```
struct block_device *
```

```
i_bdev
```

Pointer to the block device driver

```
struct cdev *
```

```
i_cdev
```

Pointer to the character device driver

```
int
```

```
i_cindex
```

Index of the device file within a group of minor numbers

10

`_ _u32`

`i_generation`

inode version number (used by some filesystems)

unsigned long

`i_dnotify_mask`

Bit mask of directory notify events

`struct dnotify_struct *`

`i_dnotify`

Used for directory notifications

unsigned long

`i_state`

inode state flags

unsigned long

`dirtied_when`

Dirtying time (in ticks) of the inode

unsigned int

`i_flags`

Filesystem mount flags

`atomic_t`

`i_writecount`

Usage counter for writing processes

`void *`

`i_security`

Pointer to inode's security structure

`void *`

`u.generic_ip`

Pointer to private data

10

`seqcount_t`

`i_size_seqcount`

Sequence counter used in SMP systems to get consistent values for `i_size`

Each inode object duplicates some of the data included in the disk inode for instance, the number of blocks allocated to the file. When the value of the `i_state` field is equal to `I_DIRTY_SYNC`, `I_DIRTY_DATASYNC`, or `I_DIRTY_PAGES`, the inode is dirty that is, the corresponding disk inode must be updated. The `I_DIRTY` macro can be used to check the value of these three flags at once (see later for details). Other values of the `i_state` field are `I_LOCK` (the inode object is involved in an I/O transfer), `I_FREEING` (the inode object is being freed), `I_CLEAR` (the inode object contents are no longer meaningful), and `I_NEW` (the inode object has been allocated but not yet filled with data read from the disk inode).

Each inode object always appears in one of the following circular doubly linked lists (in all cases, the pointers to the adjacent elements are stored in the `i_list` field):

- The list of valid unused inodes, typically those mirroring valid disk inodes and not currently used by any process. These inodes are not dirty and their `i_count` field is set to 0. The first and last elements of this list are referenced by the `next` and `prev` fields, respectively, of the `inode_unused` variable. This list acts as a disk cache.
- The list of in-use inodes, that is, those mirroring valid disk inodes and used by some process. These inodes are not dirty and their `i_count` field is positive. The first and last elements are referenced by the `inode_in_use` variable.
- The list of dirty inodes. The first and last elements are referenced by the `s_dirty` field of the corresponding superblock object.

Each of the lists just mentioned links the `i_list` fields of the proper inode objects.

Moreover, each inode object is also included in a per-filesystem doubly linked circular list headed at the `s_inodes` field of the superblock object; the `i_sb_list` field of the inode object stores the pointers for the adjacent elements in this list.

Finally, the inode objects are also included in a hash table named `inode_hashtable`. The hash table speeds up the search of the inode object when the kernel knows both the inode number and the address of the superblock object corresponding to the filesystem that includes the file. Because hashing may induce collisions, the inode object includes an `i_hash` field that contains a backward and a forward pointer to other inodes that hash to the same position; this field creates a doubly linked list of those inodes.

The methods associated with an inode object are also called inode operations. They are described by an `inode_operations` structure, whose address is included in the `i_op` field. Here are the inode operations in the order they appear in the `inode_operations` table:

`create(dir, dentry, mode, nameidata)`

Creates a new disk inode for a regular file associated with a `dentry` object in some directory.

`lookup(dir, dentry, nameidata)`

Searches a directory for an inode corresponding to the filename included in a `dentry` object.

`link(old_dentry, dir, new_dentry)`

Creates a new hard link that refers to the file specified by `old_dentry` in the directory `dir`; the new hard link has the name specified by `new_dentry`.

`unlink(dir, dentry)`

Removes the hard link of the file specified by a dentry object from a directory.

`symlink(dir, dentry, symname)`

Creates a new inode for a symbolic link associated with a dentry object in some directory.

`mkdir(dir, dentry, mode)`

Creates a new inode for a directory associated with a dentry object in some directory.

`rmdir(dir, dentry)`

Removes from a directory the subdirectory whose name is included in a dentry object.

`mknod(dir, dentry, mode, rdev)`

Creates a new disk inode for a special file associated with a dentry object in some directory. The `mode` and `rdev` parameters specify, respectively, the file type and the device's major and minor numbers.

`rename(old_dir, old_dentry, new_dir, new_dentry)`

Moves the file identified by `old_dentry` from the `old_dir` directory to the `new_dir` one. The new filename is included in the dentry object that `new_dentry` points to.

`readlink(dentry, buffer, buflen)`

Copies into a User Mode memory area specified by `buffer` the file pathname corresponding to the symbolic link specified by the dentry.

`follow_link(inode, nameidata)`

Translates a symbolic link specified by an inode object; if the symbolic link is a relative pathname, the lookup operation starts from the directory specified in the second parameter.

`put_link(dentry, nameidata)`

Releases all temporary data structures allocated by the `follow_link` method to translate a symbolic link.

`truncate(inode)`

Modifies the size of the file associated with an inode. Before invoking this method, it is necessary to set the `i_size` field of the inode object to the required new size.

`permission(inode, mask, nameidata)`

Checks whether the specified access mode is allowed for the file associated with `inode`.

`setattr(dentry, iattr)`

Notifies a "change event" after touching the inode attributes.

`getattr(mnt, dentry, kstat)`

Used by some filesystems to read inode attributes.

`setxattr(dentry, name, value, size, flags)`

Sets an "extended attribute" of an inode (extended attributes are stored on disk blocks outside of any inode).

`getxattr(dentry, name, buffer, size)`

Gets an extended attribute of an inode.

`listxattr(dentry, buffer, size)`

Gets the whole list of extended attribute names.

`removexattr(dentry, name)`

Removes an extended attribute of an inode.

The methods just listed are available to all possible inodes and filesystem types. However, only a subset of them applies to a specific inode and filesystem; the fields corresponding to unimplemented methods are set to `NULL`.

12.2.3. File Objects

A file object describes how a process interacts with a file it has opened. The object is created when the file is opened and consists of a `file` structure, whose fields are described in [Table 12-4](#). Notice that file objects have no corresponding image on disk, and hence no "dirty" field is included in the `file` structure to specify that the file object has been modified.

Table 12-4. The fields of the file object

Type

Field

Description

`struct list_head`

`f_list`

Pointers for generic file object list

```
struct dentry *
```

```
f_dentry
```

dentry object associated with the file

```
struct vfsmount *
```

```
f_vfsmnt
```

Mounted filesystem containing the file

```
struct file_operations *
```

```
f_op
```

Pointer to file operation table

```
atomic_t
```

```
f_count
```

File object's reference counter

```
unsigned int
```

```
f_flags
```

Flags specified when opening the file

```
mode_t
```

```
f_mode
```

Process access mode

```
int
```

```
f_error
```

Error code for network write operation

```
loff_t
```

```
f_pos
```

Current file offset (file pointer)

```
struct fown_struct
```

```
f_owner
```

Data for I/O event notification via signals

`unsigned int`

`f_uid`

User's UID

`unsigned int`

`f_gid`

User group ID

`struct file_ra_state`

`f_ra`

File read-ahead state (see [Chapter 16](#))

`size_t`

`f_maxcount`

Maximum number of bytes that can be read or written with a single operation (currently set to 231-1)

`unsigned long`

`f_version`

Version number, automatically increased after each use

`void *`

`f_security`

Pointer to file object's security structure

`void *`

`private_data`

Pointer to data specific for a filesystem or a device driver

`struct list_head`

`f_ep_links`

Head of the list of event poll waiters for this file

`spinlock_t`

`f_ep_lock`

Spin lock protecting the `f_ep_links` list

```
struct address_space *
f_mapping
```

Pointer to file's address space object (see [Chapter 15](#))

The main information stored in a file object is the file pointerthe current position in the file from which the next operation will take place. Because several processes may access the same file concurrently, the file pointer must be kept in the file object rather than the inode object.

File objects are allocated through a slab cache named `filp`, whose descriptor address is stored in the `filp_cache` variable. Because there is a limit on the number of file objects that can be allocated, the `files_stat` variable specifies in the `max_files` field the maximum number of allocatable file objectsi.e., the maximum number of files that can be accessed at the same time in the system.^[*]

[*] The `files_init( )` function, executed during kernel initialization, sets the `max_files` field to one-tenth of the available RAM in kilobytes, but the system administrator can tune this parameter by writing into the `/proc/sys/fs/file-max` file. Moreover, the superuser can always get a file object, even if `max_files` file objects have already been allocated.

"In use" file objects are collected in several lists rooted at the superblocks of the owning filesystems. Each superblock object stores in the `s_files` field the head of a list of file objects; thus, file objects of files belonging to different filesystems are included in different lists. The pointers to the previous and next element in the list are stored in the `f_list` field of the file object. The `files_lock` spin lock protects the superblock `s_files` lists against concurrent accesses in multiprocessor systems.

The `f_count` field of the file object is a reference counter: it counts the number of processes that are using the file object (remember however that lightweight processes created with the `CLONE_FILES` flag share the table that identifies the open files, thus they use the same file objects). The counter is also increased when the file object is used by the kernel itselffor instance, when the object is inserted in a list, or when a `dup( )` system call has been issued.

When the VFS must open a file on behalf of a process, it invokes the `get_empty_filp( )` function to allocate a new file object. The function invokes `kmem_cache_alloc( )` to get a free file object from the `filp` cache, then it initializes the fields of the object as follows:

```
memset(f, 0, sizeof(*f));
INIT_LIST_HEAD(&f->f_ep_links);
spin_lock_init(&f->f_ep_lock);
atomic_set(&f->f_count, 1);
f->f_uid = current->fsuid;
f->f_gid = current->fsgid;
f->f_owner.lock = RW_LOCK_UNLOCKED;
INIT_LIST_HEAD(&f->f_list);
f->f_maxcount = INT_MAX;
```

As we explained earlier in the section "[The Common File Model](#)," each filesystem includes its own set of file operations that perform such activities as reading and writing a file. When the kernel loads an inode into memory from disk, it stores a pointer to these file operations in a `file_operations` structure whose

address is contained in the `i_fop` field of the inode object. When a process opens the file, the VFS initializes the `f_op` field of the new file object with the address stored in the inode so that further calls to file operations can use these functions. If necessary, the VFS may later modify the set of file operations by storing a new value in `f_op`.

The following list describes the file operations in the order in which they appear in the `file_operations` table:

`llseek(file, offset, origin)`
Updates the file pointer.

`read(file, buf, count, offset)`
Reads `count` bytes from a file starting at position `*offset`; the value `*offset` (which usually corresponds to the file pointer) is then increased.

`aio_read(req, buf, len, pos)`
Starts an asynchronous I/O operation to read `len` bytes into `buf` from file position `pos` (introduced to support the `io_submit( )` system call).

`write(file, buf, count, offset)`
Writes `count` bytes into a file starting at position `*offset`; the value `*offset` (which usually corresponds to the file pointer) is then increased.

`aio_write(req, buf, len, pos)`
Starts an asynchronous I/O operation to write `len` bytes from `buf` to file position `pos`.

`readdir(dir, dirent, filldir)`
Returns the next directory entry of a directory in `dirent`; the `filldir` parameter contains the address of an auxiliary function that extracts the fields in a directory entry.

`poll(file, poll_table)`
Checks whether there is activity on a file and goes to sleep until something happens on it.

`ioctl(inode, file, cmd, arg)`
Sends a command to an underlying hardware device. This method applies only to device files.

`unlocked_ioctl(file, cmd, arg)`
Similar to the `ioctl` method, but it does not take the big kernel lock (see the section "[The Big Kernel Lock](#)" in [Chapter 5](#)). It is expected that all device drivers and all filesystems will implement this new method instead of the `ioctl` method.

`compat_ioctl(file, cmd, arg)`

Method used to implement the `ioctl()` 32-bit system call by 64-bit kernels.

`mmap(file, vma)`

Performs a memory mapping of the file into a process address space (see the section "[Memory Mapping](#)" in [Chapter 16](#)).

`open(inode, file)`

Opens a file by creating a new file object and linking it to the corresponding inode object (see the section "[The open\(\) System Call](#)" later in this chapter).

`flush(file)`

Called when a reference to an open file is closed. The actual purpose of this method is filesystem-dependent.

`release(inode, file)`

Releases the file object. Called when the last reference to an open file is closed that is, when the `f_count` field of the file object becomes 0.

`fsync(file, dentry, flag)`

Flushes the file by writing all cached data to disk.

`aio_fsync(req, flag)`

Starts an asynchronous I/O flush operation.

`fasync(fd, file, on)`

Enables or disables I/O event notification by means of signals.

`lock(file, cmd, file_lock)`

Applies a lock to the file (see the section "[File Locking](#)" later in this chapter).

`readv(file, vector, count, offset)`

Reads bytes from a file and puts the results in the buffers described by `vector`; the number of buffers is specified by `count`.

`writetv(file, vector, count, offset)`

Writes bytes into a file from the buffers described by `vector`; the number of buffers is specified by `count`.

`sendfile(in_file, offset, count, file_send_actor, out_file)`

Transfers data from `in_file` to `out_file` (introduced to support the `sendfile()` system call).

`sendpage(file, page, offset, size, pointer, fill)`

Transfers data from `file` to the page cache's `page`; this is a low-level method used by `sendfile ( )` and by the networking code for sockets.

`get_unmapped_area(file, addr, len, offset, flags)`

Gets an unused address range to map the file.

`check_flags(flags)`

Method invoked by the service routine of the `fcntl ( )` system call to perform additional checks when setting the status flags of a file (`F_SETFL` command). Currently used only by the NFS network filesystem.

`dir_notify(file, arg)`

Method invoked by the service routine of the `fcntl ( )` system call when establishing a directory change notification (`F_NOTIFY` command). Currently used only by the Common Internet File System (CIFS) network filesystem.

`flock(file, flag, lock)`

Used to customize the behavior of the `flock ( )` system call. No official Linux filesystem makes use of this method.

The methods just described are available to all possible file types. However, only a subset of them apply to a specific file type; the fields corresponding to unimplemented methods are set to `NULL`.

12.2.4. dentry Objects

We mentioned in the section "[The Common File Model](#)" that the VFS considers each directory a file that contains a list of files and other directories. We will discuss in [Chapter 18](#) how directories are implemented on a specific filesystem. Once a directory entry is read into memory, however, it is transformed by the VFS into a dentry object based on the `dentry` structure, whose fields are described in [Table 12-5](#). The kernel creates a dentry object for every component of a pathname that a process looks up; the dentry object associates the component to its corresponding inode. For example, when looking up the `/tmp/test` pathname, the kernel creates a dentry object for the `/root` directory, a second dentry object for the `tmp` entry of the root directory, and a third dentry object for the `test` entry of the `/tmp` directory.

Notice that dentry objects have no corresponding image on disk, and hence no field is included in the `dentry` structure to specify that the object has been modified. Dentry objects are stored in a slab allocator cache whose descriptor is `dentry_cache`; dentry objects are thus created and destroyed by invoking `kmem_cache_alloc ( )` and `kmem_cache_free ( )`.

Table 12-5. The fields of the dentry object

Type

Field

Description

`atomic_t`

`d_count`

Dentry object usage counter

`unsigned int`

`d_flags`

Dentry cache flags

`spinlock_t`

`d_lock`

Spin lock protecting the dentry object

`struct inode *`

`d_inode`

Inode associated with filename

`struct dentry *`

`d_parent`

Dentry object of parent directory

`struct qstr`

`d_name`

Filename

`struct list_head`

`d_lru`

Pointers for the list of unused dentries

`struct list_head`

`d_child`

For directories, pointers for the list of directory dentries in the same parent directory

`struct list_head`

`d_subdirs`

For directories, head of the list of subdirectory dentries


```
struct list_head
```

```
d_alias
```

Pointers for the list of dentries associated with the same inode (alias)

```
unsigned long
```

```
d_time
```

Used by `d_revalidate` method

```
struct dentry_operations*
```

```
d_op
```

Dentry methods

```
struct super_block *
```

```
d_sb
```

Superblock object of the file

```
void *
```

```
d_fsdata
```

Filesystem-dependent data

```
struct rcu_head
```

```
d_rcu
```

The RCU descriptor used when reclaiming the dentry object (see the section "[Read-Copy Update \(RCU\)](#)" in [Chapter 5](#))

```
struct dcookie_struct *
```

```
d_cookie
```

Pointer to structure used by kernel profilers

```
struct hlist_node
```

```
d_hash
```

Pointer for list in hash table entry

```
int
```

```
d_mounted
```

For directories, counter for the number of filesystems mounted on this dentry

`unsigned char[]`

`d_iname`

Space for short filename

Each dentry object may be in one of four states:

Free

The dentry object contains no valid information and is not used by the VFS. The corresponding memory area is handled by the slab allocator.

Unused

The dentry object is not currently used by the kernel. The `d_count` usage counter of the object is 0, but the `d_inode` field still points to the associated inode. The dentry object contains valid information, but its contents may be discarded if necessary in order to reclaim memory.

In use

The dentry object is currently used by the kernel. The `d_count` usage counter is positive, and the `d_inode` field points to the associated inode object. The dentry object contains valid information and cannot be discarded.

Negative

The inode associated with the dentry does not exist, either because the corresponding disk inode has been deleted or because the dentry object was created by resolving a pathname of a nonexistent file. The `d_inode` field of the dentry object is set to `NULL`, but the object still remains in the dentry cache, so that further lookup operations to the same file pathname can be quickly resolved. The term "negative" is somewhat misleading, because no negative value is involved.

The methods associated with a dentry object are called dentry operations ; they are described by the `dentry_operations` structure, whose address is stored in the `d_op` field. Although some filesystems define their own dentry methods, the fields are usually `NULL` and the VFS replaces them with default functions. Here are the methods, in the order they appear in the `dentry_operations` table:

`d_revalidate(dentry, nameidata)`

Determines whether the dentry object is still valid before using it for translating a file pathname. The default VFS function does nothing, although network filesystems may specify their own functions.

`d_hash(dentry, name)`

Creates a hash value; this function is a filesystem-specific hash function for the dentry hash table. The `dentry` parameter identifies the directory containing the component. The `name` parameter points to a structure containing both the pathname component to be looked up and the value produced by the hash function.

`d_compare(dir, name1, name2)`

Compares two filenames ; `name1` should belong to the directory referenced by `dir`. The default VFS function is a normal string match. However, each filesystem can implement this method in its own way. For instance, MS-DOS does not distinguish capital from lowercase letters.

`d_delete(dentry)`

Called when the last reference to a dentry object is deleted (`d_count` becomes 0). The default VFS function does nothing.

`d_release(dentry)`

Called when a dentry object is going to be freed (released to the slab allocator). The default VFS function does nothing.

`d_iput(dentry, ino)`

Called when a dentry object becomes "negative" that is, it loses its inode. The default VFS function invokes `iput ( )` to release the inode object.

12.2.5. The dentry Cache

Because reading a directory entry from disk and constructing the corresponding dentry object requires considerable time, it makes sense to keep in memory dentry objects that you've finished with but might need later. For instance, people often edit a file and then compile it, or edit and print it, or copy it and then edit the copy. In such cases, the same file needs to be repeatedly accessed.

To maximize efficiency in handling dentries, Linux uses a dentry cache, which consists of two kinds of data structures:

- A set of dentry objects in the in-use, unused, or negative state.
- A hash table to derive the dentry object associated with a given filename and a given directory quickly. As usual, if the required object is not included in the dentry cache, the search function returns a null value.

The dentry cache also acts as a controller for an inode cache . The inodes in kernel memory that are associated with unused dentries are not discarded, because the dentry cache is still using them. Thus, the inode objects are kept in RAM and can be quickly referenced by means of the corresponding dentries.

All the "unused" dentries are included in a doubly linked "Least Recently Used" list sorted by time of insertion. In other words, the dentry object that was last released is put in front of the list, so the least recently used dentry objects are always near the end of the list. When the dentry cache has to shrink, the kernel removes elements from the tail of this list so that the most recently used objects are preserved. The addresses of the first and last elements of the LRU list are stored in the `next` and `prev` fields of the `dentry_unused` variable of type `list_head`. The `d_lru` field of the dentry object contains pointers to the adjacent dentries in the list.

Each "in use" dentry object is inserted into a doubly linked list specified by the `i_dentry` field of the corresponding inode object (because each inode could be associated with several hard links, a list is required).

The `d_alias` field of the dentry object stores the addresses of the adjacent elements in the list. Both fields are of type `struct list_head`.

An "in use" dentry object may become "negative" when the last hard link to the corresponding file is deleted. In this case, the dentry object is moved into the LRU list of unused dentries. Each time the kernel shrinks the dentry cache, negative dentries move toward the tail of the LRU list so that they are gradually freed (see the section "[Reclaiming Pages of Shrinkable Disk Caches](#)" in [Chapter 17](#)).

The hash table is implemented by means of a `dentry_hashtable` array. Each element is a pointer to a list of dentries that hash to the same hash table value. The array's size usually depends on the amount of RAM installed in the system; the default value is 256 entries per megabyte of RAM. The `d_hash` field of the dentry object contains pointers to the adjacent elements in the list associated with a single hash value. The hash function produces its value from both the dentry object of the directory and the filename.

The `dcache_lock` spin lock protects the dentry cache data structures against concurrent accesses in multiprocessor systems. The `d_lookup( )` function looks in the hash table for a given parent dentry object and filename; to avoid race conditions, it makes use of a seqlock (see the section "[Seqlocks](#)" in [Chapter 5](#)). The `__d_lookup( )` function is similar, but it assumes that no race condition can happen, so it does not use the seqlock.

12.2.6. Files Associated with a Process

We mentioned in the section "[An Overview of the Unix Filesystem](#)" in [Chapter 1](#) that each process has its own current working directory and its own root directory. These are only two examples of data that must be maintained by the kernel to represent the interactions between a process and a filesystem. A whole data structure of type `fs_struct` is used for that purpose (see [Table 12-6](#)), and each process descriptor has an `fs` field that points to the process `fs_struct` structure.

Table 12-6. The fields of the `fs_struct` structure

Type

Field

Description

`atomic_t`

`count`

Number of processes sharing this table

`rwlock_t`

`lock`

Read/write spin lock for the table fields

`int`

`umask`

Bit mask used when opening the file to set the file permissions

```
struct dentry *
```

```
root
```

Dentry of the root directory

```
struct dentry *
```

```
pwd
```

Dentry of the current working directory

```
struct dentry *
```

```
altroot
```

Dentry of the emulated root directory (always NULL for the 80 x 86 architecture)

```
struct vfsmount *
```

```
rootmnt
```

Mounted filesystem object of the root directory

```
struct vfsmount *
```

```
pwdmnt
```

Mounted filesystem object of the current working directory

```
struct vfsmount *
```

```
altrootmnt
```

Mounted filesystem object of the emulated root directory (always NULL for the 80 x 86 architecture)

A second table, whose address is contained in the `files` field of the process descriptor, specifies which files are currently opened by the process. It is a `files_struct` structure whose fields are illustrated in [Table 12-7](#).

Table 12-7. The fields of the `files_struct` structure

Type

Field

Description

```
atomic_t
```

count

Number of processes sharing this table

rwlock_t

file_lock

Read/write spin lock for the table fields

int

max_fds

Current maximum number of file objects

int

max_fdset

Current maximum number of file descriptors

int

next_fd

Maximum file descriptors ever allocated plus 1

struct file **

fd

Pointer to array of file object pointers

fd_set *

close_on_exec

Pointer to file descriptors to be closed on exec ()

fd_set *

open_fds

Pointer to open file descriptors

fd_set

close_on_exec_init

Initial set of file descriptors to be closed on exec ()

fd_set

`open_fds_init`

Initial set of file descriptors

`struct file *[]`

`fd_array`

Initial array of file object pointers

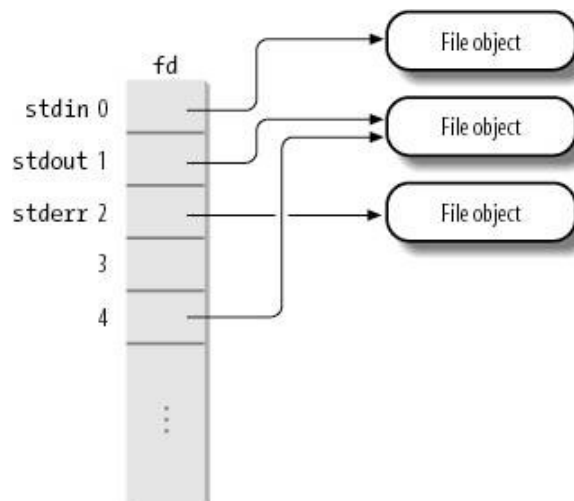
The `fd` field points to an array of pointers to file objects. The size of the array is stored in the `max_fds` field. Usually, `fd` points to the `fd_array` field of the `files_struct` structure, which includes 32 file object pointers. If the process opens more than 32 files, the kernel allocates a new, larger array of file pointers and stores its address in the `fd` fields; it also updates the `max_fds` field.

For every file with an entry in the `fd` array, the array index is the file descriptor. Usually, the first element (index 0) of the array is associated with the standard input of the process, the second with the standard output, and the third with the standard error (see [Figure 12-3](#)). Unix processes use the file descriptor as the main file identifier. Notice that, thanks to the `dup( )`, `dup2( )`, and `fcntl( )` system calls, two file descriptors may refer to the same opened file—that is, two elements of the array could point to the same file object. Users see this all the time when they use shell constructs such as `2>&1` to redirect the standard error to the standard output.

A process cannot use more than `NR_OPEN` (usually, 1,048,576) file descriptors. The kernel also enforces a dynamic bound on the maximum number of file descriptors in the `signal->rlim[RLIMIT_NOFILE]` structure of the process descriptor; this value is usually 1,024, but it can be raised if the process has root privileges.

The `open_fds` field initially contains the address of the `open_fds_init` field, which is a bitmap that identifies the file descriptors of currently opened files. The `max_fdset` field stores the number of bits in the bitmap. Because the `fd_set` data structure includes 1,024 bits, there is usually no need to expand the size of the bitmap. However, the kernel may dynamically expand the size of the bitmap if this turns out to be necessary, much as in the case of the array of file objects.

Figure 12-3. The `fd` array



The kernel provides an `fget ( )` function to be invoked when the kernel starts using a file object. This function receives as its parameter a file descriptor `fd`. It returns the address in `current->files->fd[fd]` (that is, the address of the corresponding file object), or `NULL` if no file corresponds to `fd`. In the first case, `fget ( )` increases the file object usage counter `f_count` by 1.

The kernel also provides an `fput ( )` function to be invoked when a kernel control path finishes using a file object. This function receives as its parameter the address of a file object and decreases its usage counter, `f_count`. Moreover, if this field becomes 0, the function invokes the `release` method of the file operations (if defined), decreases the `i_writecount` field in the inode object (if the file was opened for writing), removes the file object from the superblock's list, releases the file object to the slab allocator, and decreases the usage counters of the associated dentry object and of the filesystem descriptor (see the later section "[Filesystem Mounting](#)").

The `fget_light ( )` and `fput_light ( )` functions are faster versions of `fget ( )` and `fput ( )`: the kernel uses them when it can safely assume that the current process already owns the file object—that is, the process has already previously increased the file object's reference counter. For instance, they are used by the service routines of the system calls that receive a file descriptor as an argument, because the file object's reference counter has been increased by a previous `open ( )` system call.

12.3. Filesystem Types

The Linux kernel supports many different types of filesystems. In the following, we introduce a few special types of filesystems that play an important role in the internal design of the Linux kernel.

Next, we'll discuss filesystem registrationthat is, the basic operation that must be performed, usually during system initialization, before using a filesystem type. Once a filesystem is registered, its specific functions are available to the kernel, so that type of filesystem can be mounted on the system's directory tree.

12.3.1. Special Filesystems

While network and disk-based filesystems enable the user to handle information stored outside the kernel, special filesystems may provide an easy way for system programs and administrators to manipulate the data structures of the kernel and to implement special features of the operating system. [Table 12-8](#) lists the most common special filesystems used in Linux; for each of them, the table reports its suggested mount point and a short description.

Notice that a few filesystems have no fixed mount point (keyword "any" in the table). These filesystems can be freely mounted and used by the users. Moreover, some other special filesystems do not have a mount point at all (keyword "none" in the table). They are not for user interaction, but the kernel can use them to easily reuse some of the VFS layer code; for instance, we'll see in [Chapter 19](#) that, thanks to the pipefs special filesystem, pipes can be treated in the same way as FIFO files.

Table 12-8. Most common special filesystems

Name	Mount point	Description
bdev	none	Block devices (see Chapter 13)
binfmt_misc	any	Miscellaneous executable formats (see Chapter 20)
devpts	/dev/pts	Pseudoterminal support (Open Group's Unix98 standard)
eventpollfs	none	Used by the efficient event polling mechanism
futexfs	none	Used by the futex (Fast Userspace Locking) mechanism
pipefs	none	Pipes (see Chapter

		19)
proc	/proc	General access point to kernel data structures
rootfs	none	Provides an empty root directory for the bootstrap phase
shm	none	IPC-shared memory regions (see Chapter 19)
mqueue	any	Used to implement POSIX message queues (see Chapter 19)
sockfs	none	Sockets
sysfs	/sys	General access point to system data (see Chapter 13)
tmpfs	any	Temporary files (kept in RAM unless swapped)
usbfs	/proc/bus/usb	USB devices

Special filesystems are not bound to physical block devices. However, the kernel assigns to each mounted special filesystem a fictitious block device that has the value 0 as major number and an arbitrary value (different for each special filesystem) as a minor number. The `set_anon_super( )` function is used to initialize superblocks of special filesystems; this function essentially gets an unused minor number `dev` and sets the `s_dev` field of the new superblock with major number 0 and minor number `dev`. Another function called `kill_anon_super( )` removes the superblock of a special filesystem. The `unnamed_dev_idr` variable includes pointers to auxiliary structures that record the minor numbers currently in use. Although some kernel designers dislike the fictitious block device identifiers, they help the kernel to handle special filesystems and regular ones in a uniform way.

We'll see a practical example of how the kernel defines and initializes a special filesystem in the later section "[Mounting a Generic Filesystem](#)."

12.3.2. Filesystem Type Registration

Often, the user configures Linux to recognize all the filesystems needed when compiling the kernel for his system. But the code for a filesystem actually may either be included in the kernel image or dynamically loaded as a module (see [Appendix B](#)). The VFS must keep track of all filesystem types whose code is currently included in the kernel. It does this by performing filesystem type registration .

Each registered filesystem is represented as a `file_system_type` object whose fields are illustrated in [Table 12-9](#).

Table 12-9. The fields of the `file_system_type` object

Type

Field

Description

`const char *``name`

Filesystem name

`int``fs_flags`

Filesystem type flags

`struct super_block * (*)()``get_sb`

Method for reading a superblock

`void (*)()``kill_sb`

Method for removing a superblock

`struct module *``owner`Pointer to the module implementing the filesystem (see [Appendix B](#))`struct file_system_type *``next`

Pointer to the next element in the list of filesystem types

`struct list_head``fs_supers`

Head of a list of superblock objects having the same filesystem type

All filesystem-type objects are inserted into a singly linked list. The `file_systems` variable points to the first item, while the `next` field of the structure points to the next item in the list. The `file_systems_lock` read/write spin lock protects the whole list against concurrent accesses.

The `fs_supers` field represents the head (first dummy element) of a list of superblock objects corresponding to mounted filesystems of the given type. The backward and forward links of a list element are stored in the `s_instances` field of the superblock object.

The `get_sb` field points to the filesystem-type-dependent function that allocates a new superblock object and initializes it (if necessary, by reading a disk). The `kill_sb` field points to the function that destroys a superblock.

The `fs_flags` field stores several flags, which are listed in [Table 12-10](#).

Table 12-10. The filesystem type flags

Name

Description

`FS_REQUIRES_DEV`

Every filesystem of this type must be located on a physical disk device.

`FS_BINARY_MOUNTDATA`

The filesystem uses binary mount data.

`FS_REVAL_DOT`

Always revalidate the "." and ".." paths in the dentry cache (for network filesystems).

`FS_ODD_RENAME`

"Rename" operations are "move" operations (for network filesystems).

During system initialization, the `register_filesystem( )` function is invoked for every filesystem specified at compile time; the function inserts the corresponding `file_system_type` object into the filesystem-type list.

The `register_filesystem( )` function is also invoked when a module implementing a filesystem is loaded. In this case, the filesystem may also be unregistered (by invoking the `unregister_filesystem( )` function) when the module is unloaded.

The `get_fs_type( )` function, which receives a filesystem name as its parameter, scans the list of registered filesystems looking at the `name` field of their descriptors, and returns a pointer to the corresponding `file_system_type` object, if it is present.

12.4. Filesystem Handling

Like every traditional Unix system, Linux makes use of a system's root filesystem : it is the filesystem that is directly mounted by the kernel during the booting phase and that holds the system initialization scripts and the most essential system programs.

Other filesystems can be mounted either by the initialization scripts or directly by the user on directories of already mounted filesystems. Being a tree of directories, every filesystem has its own root directory. The directory on which a filesystem is mounted is called the mount point. A mounted filesystem is a child of the mounted filesystem to which the mount point directory belongs. For instance, the */proc* virtual filesystem is a child of the system's root filesystem (and the system's root filesystem is the parent of */proc*). The root directory of a mounted filesystem hides the content of the mount point directory of the parent filesystem, as well as the whole subtree of the parent filesystem below the mount point.^[*]

[*] The root directory of a filesystem can be different from the root directory of a process: as we have seen in the earlier section "[Files Associated with a Process](#)," the process's root directory is the directory corresponding to the "/" pathname. By default, the process' root directory coincides with the root directory of the system's root filesystem (or more precisely, with the root directory of the root filesystem in the namespace of the process, described in the following section), but it can be changed by invoking the `chroot ( )` system call.

12.4.1. Namespaces

In a traditional Unix system, there is only one tree of mounted filesystems: starting from the system's root filesystem, each process can potentially access every file in a mounted filesystem by specifying the proper pathname. In this respect, Linux 2.6 is more refined: every process might have its own tree of mounted filesystems the so-called namespace of the process.

Usually most processes share the same namespace, which is the tree of mounted filesystems that is rooted at the system's root filesystem and that is used by the `init` process. However, a process gets a new namespace if it is created by the `clone ( )` system call with the `CLONE_NEWNS` flag set (see the section "[The clone\(\), fork\(\), and vfork\(\) System Calls](#)" in [Chapter 3](#)). The new namespace is then inherited by children processes if the parent creates them without the `CLONE_NEWNS` flag.

When a process mounts or unmounts a filesystem, it only modifies its namespace. Therefore, the change is visible to all processes that share the same namespace, and only to them. A process can even change the root filesystem of its namespace by using the Linux-specific `pivot_root ( )` system call.

The namespace of a process is represented by a `namespace` structure pointed to by the `namespace` field of the process descriptor. The fields of the `namespace` structure are shown in Table 12-11.

Table 12-11. The fields of the namespace structure

Type	Field	Description
<code>atomic_t</code>	<code>count</code>	Usage counter (how many processes share the namespace)
		Mounted filesystem
<code>struct vfsmount *</code>	<code>root</code>	descriptor for the root directory of the namespace
<code>struct list_head</code>	<code>list</code>	Head of list of all mounted filesystem descriptors
<code>struct rw_semaphore</code>	<code>sem</code>	Read/write semaphore protecting this structure

The `list` field is the head of a doubly linked circular list collecting all mounted filesystems that belong to the namespace. The `root` field specifies the mounted filesystem that represents the root of the tree of mounted filesystems of this namespace. As we will see in the next section, mounted filesystems are represented by `vfsmount` structures.

12.4.2. Filesystem Mounting

In most traditional Unix-like kernels, each filesystem can be mounted only once. Suppose that an Ext2 filesystem stored in the `/dev/fd0` floppy disk is mounted on `/flp` by issuing the command:

```
mount -t ext2 /dev/fd0 /flp
```

Until the filesystem is unmounted by issuing a `umount` command, every other mount command acting on `/dev/fd0` fails.

However, Linux is different: it is possible to mount the same filesystem several times. Of course, if a filesystem is mounted `n` times, its root directory can be accessed through `n` mount points, one per mount operation. Although the same filesystem can be accessed by using different mount points, it is really unique. Thus, there is only one superblock object for all of them, no matter of how many times it has been mounted.

Mounted filesystems form a hierarchy: the mount point of a filesystem might be a directory of a second filesystem, which in turn is already mounted over a third filesystem, and so on.^[*]

[*] Quite surprisingly, the mount point of a filesystem might be a directory of the same filesystem, provided that it was already mounted. For instance:

```
mount -t ext2 /dev/fd0 /flp; touch /flp/foo
mkdir /flp/mnt; mount -t ext2 /dev/fd0 /flp/mnt
```

Now, the empty foo file on the floppy filesystem can be accessed both as /flp/foo and /flp/mnt/foo.

It is also possible to stack multiple mounts on a single mount point. Each new mount on the same mount point hides the previously mounted filesystem, although processes already using the files and directories under the old mount can continue to do so. When the topmost mounting is removed, then the next lower mount is once more made visible.

As you can imagine, keeping track of mounted filesystems can quickly become a nightmare. For each mount operation, the kernel must save in memory the mount point and the mount flags, as well as the relationships between the filesystem to be mounted and the other mounted filesystems. Such information is stored in a mounted filesystem descriptor of type `vfsmount`. The fields of this descriptor are shown in [Table 12-12](#).

Table 12-12. The fields of the `vfsmount` data structure

Type

Field

Description

`struct list_head`

`mnt_hash`

Pointers for the hash table list.

`struct vfsmount *`

`mnt_parent`

Points to the parent filesystem on which this filesystem is mounted.

`struct dentry *`

`mnt_mountpoint`

Points to the `dentry` of the mount point directory where the filesystem is mounted.

`struct dentry *`

`mnt_root`

Points to the `dentry` of the root directory of this filesystem.

`struct super_block *`

`mnt_sb`

Points to the superblock object of this filesystem.

`struct list_head`

`mnt_mounts`

Head of a list including all filesystem descriptors mounted on directories of this filesystem.

`struct list_head`

`mnt_child`

Pointers for the `mnt_mounts` list of mounted filesystem descriptors.

`atomic_t`

`mnt_count`

Usage counter (increased to forbid filesystem unmounting).

`int`

`mnt_flags`

Flags.

`int`

`mnt_expiry_mark`

Flag set to true if the filesystem is marked as expired (the filesystem can be automatically unmounted if the flag is set and no one is using it).

`char *`

`mnt_devname`

Device filename.

`struct list_head`

`mnt_list`

Pointers for namespace's list of mounted filesystem descriptors.

`struct list_head`

`mnt_fslink`

Pointers for the filesystem-specific expire list.

`struct namespace *`

`mnt_namespace`

Pointer to the namespace of the process that mounted the filesystem.

The `vfsmount` data structures are kept in several doubly linked circular lists:

- A hash table indexed by the address of the `vfsmount` descriptor of the parent filesystem and the address of the dentry object of the mount point directory. The hash table is stored in the `mount_hashtable` array, whose size depends on the amount of RAM in the system. Each item of the table is the head of a circular doubly linked list storing all descriptors that have the same hash value. The `mnt_hash` field of the descriptor contains the pointers to adjacent elements in this list.
- For each namespace, a circular doubly linked list including all mounted filesystem descriptors belonging to the namespace. The `list` field of the `namespace` structure stores the head of the list, while the `mnt_list` field of the `vfsmount` descriptor contains the pointers to adjacent elements in the list.
- For each mounted filesystem, a circular doubly linked list including all child mounted filesystems. The head of each list is stored in the `mnt_mounts` field of the mounted filesystem descriptor; moreover, the `mnt_child` field of the descriptor stores the pointers to the adjacent elements in the list.

The `vfsmount_lock` spin lock protects the lists of mounted filesystem objects from concurrent accesses.

The `mnt_flags` field of the descriptor stores the value of several flags that specify how some kinds of files in the mounted filesystem are handled. These flags, which can be set through options of the *mount* command, are listed in [Table 12-13](#).

Table 12-13. Mounted filesystem flags

Name

Description

`MNT_NOSUID`

Forbid `setuid` and `setgid` flags in the mounted filesystem

`MNT_NODEV`

Forbid access to device files in the mounted filesystem

`MNT_NOEXEC`

Disallow program execution in the mounted filesystem

Here are some functions that handle the mounted filesystem descriptors:

`alloc_vfsmnt(name)`

Allocates and initializes a mounted filesystem descriptor

`free_vfsmnt(mnt)`

Frees a mounted filesystem descriptor pointed to by `mnt`

`lookup_mnt(mnt, dentry)`

Looks up a descriptor in the hash table and returns its address

12.4.3. Mounting a Generic Filesystem

We'll now describe the actions performed by the kernel in order to mount a filesystem. We'll start by considering a filesystem that is going to be mounted over a directory of an already mounted filesystem (in this discussion we will refer to this new filesystem as "generic").

The `mount ( )` system call is used to mount a generic filesystem; its `sys_mount ( )` service routine acts on the following parameters:

- The pathname of a device file containing the filesystem, or `NULL` if it is not required (for instance, when the filesystem to be mounted is network-based)
- The pathname of the directory on which the filesystem will be mounted (the mount point)
- The filesystem type, which must be the name of a registered filesystem
- The mount flags (permitted values are listed in [Table 12-14](#))
- A pointer to a filesystem-dependent data structure (which may be `NULL`)

Table 12-14. Flags used by the `mount()` system call

Macro

Description

`MS_RDONLY`

Files can only be read

`MS_NOSUID`

Forbid `setuid` and `setgid` flags

`MS_NODEV`

Forbid access to device files

`MS_NOEXEC`

Disallow program execution

`MS_SYNCHRONOUS`

Write operations on files and directories are immediate

MS_REMOUNT

Remount the filesystem changing the mount flags

MS_MANDLOCK

Mandatory locking allowed

MS_DIRSYNC

Write operations on directories are immediate

MS_NOATIME

Do not update file access time

MS_NODIRATIME

Do not update directory access time

MS_BIND

Create a "bind mount," which allows making a file or directory visible at another point of the system directory tree (option `--bind` of the mount command)

MS_MOVE

Atomically move a mounted filesystem to another mount point (option `--move` of the mount command)

MS_REC

Recursively create "bind mounts" for a directory subtree

MS_VERBOSE

Generate kernel messages on mount errors

The `sys_mount ( )` function copies the value of the parameters into temporary kernel buffers, acquires the big kernel lock, and invokes the `do_mount ( )` function. Once `do_mount ( )` returns, the service routine releases the big kernel lock and frees the temporary kernel buffers.

The `do_mount ( )` function takes care of the actual mount operation by performing the following operations:

1. If some of the `MS_NOSUID`, `MS_NODEV`, or `MS_NOEXEC` mount flags are set, it clears them and sets the corresponding flag (`MNT_NOSUID`, `MNT_NODEV`, `MNT_NOEXEC`) in the mounted filesystem object.
2. Looks up the pathname of the mount point by invoking `path_lookup ( )`; this function stores the result of the pathname lookup in the local variable `nd` of type `nameidata` (see the later section "[Pathname Lookup](#)").
3. Examines the mount flags to determine what has to be done. In particular:

- a. If the `MS_REMOUNT` flag is specified, the purpose is usually to change the mount flags in the `s_flags` field of the superblock object and the mounted filesystem flags in the `mnt_flags` field of the mounted filesystem object. The `do_remount ( )` function performs these changes.
 - b. Otherwise, it checks the `MS_BIND` flag. If it is specified, the user is asking to make visible a file or directory on another point of the system directory tree.
 - c. Otherwise, it checks the `MS_MOVE` flag. If it is specified, the user is asking to change the mount point of an already mounted filesystem. The `do_move_mount ( )` function does this atomically.
 - d. Otherwise, it invokes `do_new_mount ( )`. This is the most common case. It is triggered when the user asks to mount either a special filesystem or a regular filesystem stored in a disk partition. `do_new_mount ( )` invokes the `do_kern_mount ( )` function passing to it the filesystem type, the mount flags, and the block device name. This function, which takes care of the actual mount operation and returns the address of a new mounted filesystem descriptor, is described below. Next, `do_new_mount ( )` invokes `do_add_mount ( )`, which essentially performs the following actions:
 1. Acquires for writing the `namespace->sem` semaphore of the current process, because the function is going to modify the namespace.
 2. The `do_kern_mount ( )` function might put the current process to sleep; meanwhile, another process might mount a filesystem on the very same mount point as ours or even change our root filesystem (`current->namespace->root`). Verifies that the lastly mounted filesystem on this mount point still refers to the current's namespace; if not, releases the read/write semaphore and returns an error code.
 3. If the filesystem to be mounted is already mounted on the mount point specified as parameter of the system call, or if the mount point is a symbolic link, it releases the read/write semaphore and returns an error code.
 4. Initializes the flags in the `mnt_flags` field of the new mounted filesystem object allocated by `do_kern_mount ( )`.
 5. Invokes `graft_tree ( )` to insert the new mounted filesystem object in the namespace list, in the hash table, and in the children list of the parent-mounted filesystem.
 6. Releases the `namespace->sem` read/write semaphore and returns.
4. Invokes `path_release ( )` to terminate the pathname lookup of the mount point (see the later section "[Pathname Lookup](#)") and returns 0.

12.4.3.1. The `do_kern_mount ( )` function

The core of the mount operation is the `do_kern_mount ( )` function, which checks the filesystem type flags to determine how the mount operation is to be done. This function receives the following parameters:

`fstype`

The name of the filesystem type to be mounted

`flags`

The mount flags (see [Table 12-14](#))

name

The pathname of the block device storing the filesystem (or the filesystem type name for special filesystems)

data

Pointer to additional data to be passed to the `read_super` method of the filesystem

The function takes care of the actual mount operation by performing essentially the following operations:

1. Invokes `get_fs_type( )` to search in the list of filesystem types and locate the name stored in the `fstype` parameter; `get_fs_type( )` returns in the local variable `type` the address of the corresponding `file_system_type` descriptor.
2. Invokes `alloc_vfsmnt( )` to allocate a new mounted filesystem descriptor and stores its address in the `mnt` local variable.
3. Invokes the `type->get_sb( )` filesystem-dependent function to allocate a new superblock and to initialize it (see below).
4. Initializes the `mnt->mnt_sb` field with the address of the new superblock object.
5. Initializes the `mnt->mnt_root` field with the address of the dentry object corresponding to the root directory of the filesystem, and increases the usage counter of the dentry object.
6. Initializes the `mnt->mnt_parent` field with the value in `mnt` (for generic filesystems, the proper value of `mnt_parent` will be set when the mounted filesystem descriptor is inserted in the proper lists by `graft_tree( )`; see step 3d5 of `do_mount( )`).
7. Initializes the `mnt->mnt_namespace` field with the value in `current->namespace`.
8. Releases the `s_umount` read/write semaphore of the superblock object (it was acquired when the object was allocated in step 3).
9. Returns the address `mnt` of the mounted filesystem object.

12.4.3.2. Allocating a superblock object

The `get_sb` method of the filesystem object is usually implemented by a one-line function. For instance, in the Ext2 filesystem the method is implemented as follows:

```
struct super_block * ext2_get_sb(struct file_system_type *type,
                                int flags, const char *dev_name, void *data)
{
    return get_sb_bdev(type, flags, dev_name, data, ext2_fill_super);
}
```

The `get_sb_bdev( )` VFS function allocates and initializes a new superblock suitable for disk-based filesystems ; it receives the address of the `ext2_fill_super( )` function, which reads the disk superblock from the Ext2 disk partition.

To allocate superblocks suitable for special filesystems , the VFS also provides the `get_sb_pseudo( )` function (for special filesystems with no mount point such as pipefs), the `get_sb_single( )` function (for special filesystems with single mount point such as sysfs), and the `get_sb_nodev( )` function (for special filesystems that can be mounted several times such as tmpfs ; see below).

The most important operations performed by `get_sb_bdev( )` are the following:

1. Invokes `open_bdev_excl( )` to open the block device having device file name `dev_name` (see the section "[Character Device Drivers](#)" in [Chapter 13](#)).
2. Invokes `sget( )` to search the list of superblock objects of the filesystem (`type->fs_supers`, see the earlier section "[Filesystem Type Registration](#)"). If a superblock relative to the block device is already present, the function returns its address. Otherwise, it allocates and initializes a new superblock object, inserts it into the filesystem list and in the global list of superblocks, and returns its address.
3. If the superblock is not new (it was not allocated in the previous step, because the filesystem is already mounted), it jumps to step 6.
4. Copies the value of the `flags` parameter into the `s_flags` field of the superblock and sets the `s_id`, `s_old_blocksize`, and `s_blocksize` fields with the proper values for the block device.
5. Invokes the filesystem-dependent function passed as last argument to `get_sb_bdev( )` to access the superblock information on disk and fill the other fields of the new superblock object.
6. Returns the address of the new superblock object.

12.4.4. Mounting the Root Filesystem

Mounting the root filesystem is a crucial part of system initialization. It is a fairly complex procedure, because the Linux kernel allows the root filesystem to be stored in many different places, such as a hard disk partition, a floppy disk, a remote filesystem shared via NFS, or even a ramdisk (a fictitious block device kept in RAM).

To keep the description simple, let's assume that the root filesystem is stored in a partition of a hard disk (the most common case, after all). While the system boots, the kernel finds the major number of the disk that contains the root filesystem in the `ROOT_DEV` variable (see [Appendix A](#)). The root filesystem can be specified as a device file in the `/dev` directory either when compiling the kernel or by passing a suitable "root" option to the initial bootstrap loader. Similarly, the mount flags of the root filesystem are stored in the `root_mountflags` variable. The user specifies these flags either by using the `rdev` external program on a compiled kernel image or by passing a suitable `rootflags` option to the initial bootstrap loader (see [Appendix A](#)).

Mounting the root filesystem is a two-stage procedure, shown in the following list:

1. The kernel mounts the special `rootfs` filesystem, which simply provides an empty directory that serves as initial mount point.
2. The kernel mounts the real root filesystem over the empty directory.

Why does the kernel bother to mount the `rootfs` filesystem before the real one? Well, the `rootfs` filesystem allows the kernel to easily change the real root filesystem. In fact, in some cases, the kernel mounts and unmounts several root filesystems, one after the other. For instance, the initial bootstrap CD of a distribution might load in RAM a kernel with a minimal set of drivers, which mounts as root a minimal filesystem stored in a ramdisk. Next, the programs in this initial root filesystem probe the hardware of the system (for instance, they determine whether the hard disk is EIDE, SCSI, or whatever), load all needed kernel modules, and remount the root filesystem from a physical block device.

12.4.4.1. Phase 1: Mounting the `rootfs` filesystem

The first stage is performed by the `init_rootfs( )` and `init_mount_tree( )` functions, which are executed during system initialization.

The `init_rootfs( )` function registers the special filesystem type `rootfs`:

```
struct file_system_type rootfs_fs_type = {
    .name = "rootfs";
    .get_sb = rootfs_get_sb;
    .kill_sb = kill_litter_super;
};
register_filesystem(&rootfs_fs_type);
```

The `init_mount_tree( )` function executes the following operations:

1. Invokes `do_kern_mount( )` passing to it the string `"rootfs"` as filesystem type, and stores the address of the mounted filesystem descriptor returned by this function in the `mnt` local variable. As explained in the previous section, `do_kern_mount( )` ends up invoking the `get_sb` method of the `rootfs` filesystem, that is, the `rootfs_get_sb( )` function:

```
struct superblock *rootfs_get_sb(struct file_system_type *fs_type,
                                int flags, const char *dev_name, void *data)
{
    return get_sb_nodev(fs_type, flags|MS_NOUSER, data,
                       ramfs_fill_super);
}
```

The `get_sb_nodev( )` function, in turn, executes the following steps:

- a. Invokes `sget( )` to allocate a new superblock passing as parameter the address of the `set_anon_super( )` function (see the earlier section ["Special Filesystems"](#)). As a result, the `s_dev` field of the superblock is set in the appropriate way: major number 0, minor number different from those of other mounted special filesystems.
 - b. Copies the value of the `flags` parameter into the `s_flags` field of the superblock.
 - c. Invokes `ramfs_fill_super( )` to allocate an inode object and a corresponding dentry object, and to fill the superblock fields. Because `rootfs` is a special filesystem that has no disk superblock, only a couple of superblock operations need to be implemented.
 - d. Returns the address of the new superblock.
2. Allocates a namespace object for the namespace of process 0, and inserts into it the mounted filesystem descriptor returned by `do_kern_mount( )`:

```
namespace = kmalloc(sizeof(*namespace), GFP_KERNEL);
list_add(&mnt->mnt_list, &namespace->list);
namespace->root = mnt;
mnt->mnt_namespace = init_task.namespace = namespace;
```

3. Sets the namespace field of every other process in the system to the address of the namespace object; also initializes the `namespace->count` usage counter. (By default, all processes share the same, initial namespace.)
4. Sets the root directory and the current working directory of process 0 to the root filesystem.

12.4.4.2. Phase 2: Mounting the real root filesystem

The second stage of the mount operation for the root filesystem is performed by the kernel near the end of the system initialization. There are several ways to mount the real root filesystem, according to the options selected when the kernel has been compiled and to the boot options passed by the kernel loader. For the sake of brevity, we consider the case of a disk-based filesystem whose device file name has been passed to the kernel by means of the "root" boot parameter. We also assume that no initial special filesystem is used, except the rootfs filesystem.

The `prepare_namespace ( )` function executes the following operations:

1. Sets the `root_device_name` variable with the device filename obtained from the "root" boot parameter. Also, sets the `ROOT_DEV` variable with the major and minor numbers of the same device file.
2. Invokes the `mount_root ( )` function, which in turn:
 - a. Invokes `sys_mknod ( )` (the service routine of the `mknod ( )` system call) to create a `/dev/root` device file in the rootfs initial root filesystem, having the major and minor numbers as in `ROOT_DEV`.
 - b. Allocates a buffer and fills it with a list of filesystem type names. This list is either passed to the kernel in the "rootfstype" boot parameter or built by scanning the elements in the singly linked list of filesystem types.
 - c. Scans the list of filesystem type names built in the previous step. For each name, it invokes `sys_mount ( )` to try to mount the given filesystem type on the root device. Because each filesystem-specific method uses a different magic number, all `get_sb ( )` invocations will fail except the one that attempts to fill the superblock by using the function of the filesystem really used on the root device. The filesystem is mounted on a directory named `/root` of the rootfs filesystem.
 - d. Invokes `sys_chdir ("/root")` to change the current directory of the process.
3. Moves the mount point of the mounted filesystem on the root directory of the rootfs filesystem:

```
sys_mount(".", "/", NULL, MS_MOVE, NULL);
sys_chroot(".");
```

Notice that the rootfs special filesystem is not unmounted: it is only hidden under the disk-based root filesystem.

12.4.5. Unmounting a Filesystem

The `umount ( )` system call is used to unmount a filesystem. The corresponding `sys_umount ( )` service routine acts on two parameters: a filename (either a mount point directory or a block device filename) and a set of flags. It performs the following actions:

1. Invokes `path_lookup ( )` to look up the mount point pathname; this function returns the results of the lookup operation in a local variable `nd` of type `nameidata` (see next section).
2. If the resulting directory is not the mount point of a filesystem, it sets the `retval` return code to `-EINVAL` and jumps to step 6. This check is done by verifying that `nd->mnt->mnt_root` contains the address of the dentry object pointed to by `nd.dentry`.
3. If the filesystem to be unmounted has not been mounted in the namespace, it sets the `retval` return code to `-EINVAL` and jumps to step 6. (Recall that some special filesystems have no mount point.) This check is done by invoking the `check_mnt ( )` function on `nd->mnt`.

4. If the user does not have the privileges required to unmount the filesystem, it sets the `retval` return code to `-EPERM` and jumps to step 6.
5. Invokes `do_umount ( )` passing as parameters `nd.mnt` (the mounted filesystem object) and `flags` (the set of flags). This function performs essentially the following operations:
 - a. Retrieves the address of the `sb` superblock object from the `mnt_sb` field of the mounted filesystem object.
 - b. If the user asked to force the unmount operation, it interrupts any ongoing mount operation by invoking the `umount_begin` superblock operation.
 - c. If the filesystem to be unmounted is the root filesystem and the user didn't ask to actually detach it, it invokes `do_remount_sb ( )` to remount the root filesystem read-only and terminates.
 - d. Acquires for writing the `namespace->sem` read/write semaphore of the current process, and gets the `vfsmount_lock` spin lock.
 - e. If the mounted filesystem does not include mount points for any child mounted filesystem, or if the user asked to forcibly detach the filesystem, it invokes `umount_tree ( )` to unmount the filesystem (together with all children filesystems).
 - f. Releases the `vfsmount_lock` spin lock and the `namespace->sem` read/write semaphore of the current process.
6. Decreases the usage counters of the `dentry` object corresponding to the root directory of the filesystem and of the mounted filesystem descriptor; these counters were increased by `path_lookup ( )`.
7. Returns the `retval` value.

12.5. Pathname Lookup

When a process must act on a file, it passes its file pathname to some VFS system call, such as `open( )`, `mkdir( )`, `rename( )`, or `stat( )`. In this section, we illustrate how the VFS performs a pathname lookup, that is, how it derives an inode from the corresponding file pathname.

The standard procedure for performing this task consists of analyzing the pathname and breaking it into a sequence of filenames. All filenames except the last must identify directories.

If the first character of the pathname is `/`, the pathname is absolute, and the search starts from the directory identified by `current->fs->root` (the process root directory). Otherwise, the pathname is relative, and the search starts from the directory identified by `current->fs->pwd` (the process-current directory).

Having in hand the dentry, and thus the inode, of the initial directory, the code examines the entry matching the first name to derive the corresponding inode. Then the directory file that has that inode is read from disk and the entry matching the second name is examined to derive the corresponding inode. This procedure is repeated for each name included in the path.

The dentry cache considerably speeds up the procedure, because it keeps the most recently used dentry objects in memory. As we saw before, each such object associates a filename in a specific directory to its corresponding inode. In many cases, therefore, the analysis of the pathname can avoid reading the intermediate directories from disk.

However, things are not as simple as they look, because the following Unix and VFS filesystem features must be taken into consideration:

- The access rights of each directory must be checked to verify whether the process is allowed to read the directory's content.
- A filename can be a symbolic link that corresponds to an arbitrary pathname; in this case, the analysis must be extended to all components of that pathname.
- Symbolic links may induce circular references; the kernel must take this possibility into account and break endless loops when they occur.
- A filename can be the mount point of a mounted filesystem. This situation must be detected, and the lookup operation must continue into the new filesystem.

- Pathname lookup has to be done inside the namespace of the process that issued the system call. The same pathname used by two processes with different namespaces may specify different files.

Pathname lookup is performed by the `path_lookup( )` function, which receives three parameters:

`name`

A pointer to the file pathname to be resolved.

`flags`

The value of flags that represent how the looked-up file is going to be accessed. The allowed values are included later in [Table 12-16](#).

`nd`

The address of a `nameidata` data structure, which stores the results of the lookup operation and whose fields are shown in [Table 12-15](#).

When `path_lookup( )` returns, the `nameidata` structure pointed to by `nd` is filled with data pertaining to the pathname lookup operation.

Table 12-15. The fields of the `nameidata` data structure

Type	Field	Description
<code>struct dentry *</code>	<code>dentry</code>	Address of the dentry object
<code>struct vfs_mount *</code>	<code>mnt</code>	Address of the mounted filesystem object
<code>struct qstr</code>	<code>last</code>	Last component of the pathname (used when the <code>LOOKUP_PARENT</code> flag is set)
<code>unsigned int</code>	<code>flags</code>	Lookup flags
<code>int</code>	<code>last_type</code>	Type of last component of the pathname (used when the <code>LOOKUP_PARENT</code> flag is set)
<code>unsigned int</code>	<code>depth</code>	Current level of symbolic link nesting (see below); it must be smaller than 6
<code>char[] *</code>	<code>saved_names</code>	Array of pathnames associated with nested symbolic links
<code>union</code>	<code>intent</code>	One-member union specifying how the file will be accessed

The `dentry` and `mnt` fields point respectively to the `dentry` object and the mounted filesystem object of the last resolved component in the `pathname`. These two fields "describe" the file that is identified by the given `pathname`.

Because the `dentry` object and the mounted filesystem object returned by the `path_lookup( )` function in the `nameidata` structure represent the result of a lookup operation, both objects should not be freed until the caller of `path_lookup( )` finishes using them. Therefore, `path_lookup( )` increases the usage counters of both objects. If the caller wants to release these objects, it invokes the `path_release( )` function passing as parameter the address of a `nameidata` structure.

The `flags` field stores the value of some flags used in the lookup operation; they are listed in [Table 12-16](#). Most of these flags can be set by the caller in the `flags` parameter of `path_lookup( )`.

Table 12-16. The flags of the lookup operation

Macro

Description

`LOOKUP_FOLLOW`

If the last component is a symbolic link, interpret (follow) it

`LOOKUP_DIRECTORY`

The last component must be a directory

`LOOKUP_CONTINUE`

There are still filenames to be examined in the `pathname`

`LOOKUP_PARENT`

Look up the directory that includes the last component of the `pathname`

`LOOKUP_NOALT`

Do not consider the emulated root directory (useless in the 80x86 architecture)

`LOOKUP_OPEN`

Intent is to open a file

`LOOKUP_CREATE`

Intent is to create a file (if it doesn't exist)

`LOOKUP_ACCESS`

Intent is to check user's permission for a file

The `path_lookup( )` function executes the following steps:

1. Initializes some fields of the `nd` parameter as follows:
 - a. Sets the `last_type` field to `LAST_ROOT` (this is needed if the pathname is a slash or a sequence of slashes; see the later section "[Parent Pathname Lookup](#)").
 - b. Sets the `flags` field to the value of the `flags` parameter
 - c. Sets the `depth` field to 0.
2. Acquires for reading the `current->fs->lock` read/write semaphore of the current process.
3. If the first character in the pathname is a slash (/), the lookup operation must start from the root directory of `current`: the function gets the addresses of the corresponding mounted filesystem object (`current->fs->rootmnt`) and dentry object (`current->fs->root`), increases their usage counters, and stores the addresses in `nd->mnt` and `nd->dentry`, respectively.
4. Otherwise, if the first character in the pathname is not a slash, the lookup operation must start from the current working directory of `current`: the function gets the addresses of the corresponding mounted filesystem object (`current->fs->pwdmnt`) and dentry object (`current->fs->pwd`), increases their usage counters, and stores the addresses in `nd->mnt` and `nd->dentry`, respectively.
5. Releases the `current->fs->lock` read/write semaphore of the current process.
6. Sets the `total_link_count` field in the descriptor of the current process to 0 (see the later section "[Lookup of Symbolic Links](#)").
7. Invokes the `link_path_walk( )` function to take care of the undergoing lookup operation:

```
return link_path_walk(name, nd);
```

We are now ready to describe the core of the pathname lookup operation, namely the `link_path_walk( )` function. It receives as its parameters a pointer `name` to the pathname to be resolved and the address `nd` of a `nameidata` data structure.

To make things a bit easier, we first describe what `link_path_walk( )` does when `LOOKUP_PARENT` is not set and the pathname does not contain symbolic links (standard pathname lookup). Next, we discuss the case in which `LOOKUP_PARENT` is set: this type of lookup is required when creating, deleting, or renaming a directory entry, that is, during a parent pathname lookup. Finally, we explain how the function resolves symbolic links.

12.5.1. Standard Pathname Lookup

When the `LOOKUP_PARENT` flag is cleared, `link_path_walk( )` performs the following steps.

1. Initializes the `lookup_flags` local variable with `nd->flags`.
2. Skips all leading slashes (/) before the first component of the pathname.
3. If the remaining pathname is empty, it returns the value 0. In the `nameidata` data structure, the `dentry` and `mnt` fields point to the objects relative to the last resolved component of the original pathname.
4. If the `depth` field of the `nd` descriptor is positive, it sets the `LOOKUP_FOLLOW` flag in the `lookup_flags` local variable (see the section "[Lookup of Symbolic Links](#)").
5. Executes a cycle that breaks the pathname passed in the `name` parameter into components (the intermediate slashes are treated as filename separators); for each component found, the function:
 - a. Retrieves the address of the inode object of the last resolved component from `nd->dentry->d_inode`. (In the first iteration, the inode refers to the directory from where to start the pathname lookup.)

- b. Checks that the permissions of the last resolved component stored into the inode allow execution (in Unix, a directory can be traversed only if it is executable). If the inode has a custom permission method, the function executes it; otherwise, it executes the `exec_permission_lite( )` function, which examines the access mode stored in the `i_mode` inode field and the privileges of the running process. In both cases, if the last resolved component does not allow execution, `link_path_walk( )` breaks out of the cycle and returns an error code.
- c. Considers the next component to be resolved. From its name, the function computes a 32-bit hash value to be used when looking in the dentry cache hash table.
- d. Skips any trailing slash (/) after the slash that terminates the name of the component to be resolved.
- e. If the component to be resolved is the last one in the original pathname, it jumps to step 6.
- f. If the name of the component is "." (a single dot), it continues with the next component ("." refers to the current directory, so it has no effect inside a pathname).
- g. If the name of the component is ".." (two dots), it tries to climb to the parent directory:
 1. If the last resolved directory is the process's root directory (`nd->dentry` is equal to `current->fs->root` and `nd->mnt` is equal to `current->fs->rootmnt`), then climbing is not allowed: it invokes `follow_mount( )` on the last resolved component (see below) and continues with the next component.
 2. If the last resolved directory is the root directory of the `nd->mnt` filesystem (`nd->dentry` is equal to `nd->mnt->mnt_root`) and the `nd->mnt` filesystem is not mounted on top of another filesystem (`nd->mnt` is equal to `nd->mnt->mnt_parent`), then the `nd->mnt` filesystem is usually^[*] the namespace's root filesystem: in this case, climbing is impossible, thus invokes `follow_mount( )` on the last resolved component (see below) and continues with the next component.

[*] This case can also occur for network filesystems disconnected from the namespace's directory tree.

3. If the last resolved directory is the root directory of the `nd->mnt` filesystem and the `nd->mnt` filesystem is mounted on top of another filesystem, a filesystem switch is required. So, the function sets `nd->dentry` to `nd->mnt->mnt_mountpoint`, and `nd->mnt` to `nd->mnt->mnt_parent`, then restarts step 5g (recall that several filesystems can be mounted on the same mount point).
4. If the last resolved directory is not the root directory of a mounted filesystem, then the function must simply climb to the parent directory: it sets `nd->dentry` to `nd->dentry->d_parent`, invokes `follow_mount( )` on the parent directory, and continues with the next component.

The `follow_mount( )` function checks whether `nd->dentry` is a mount point for some filesystem (`nd->dentry->d_mounted` is greater than zero); in this case, it invokes `lookup_mnt( )` to search the root directory of the mounted filesystem in the dentry cache, and updates `nd->dentry` and `nd->mnt` with the object addresses corresponding to the mounted filesystem; then, it repeats the whole operation (there can be several filesystems mounted on the same mount point). Essentially, invoking the `follow_mount( )` function when climbing to the parent directory is required because the process could start the pathname lookup from a directory included in a filesystem hidden by another filesystem mounted over the parent directory.

- h. The component name is neither "." nor "..", so the function must look it up in the dentry cache. If the low-level filesystem has a custom `d_hash` dentry method, the function invokes it to modify the hash value already computed in step 5c.
- i. Sets the `LOOKUP_CONTINUE` flag in `nd->flags` to denote that there is a next component to be analyzed.
- j. Invokes `do_lookup( )` to derive the dentry object associated with a given parent directory

(`nd->dentry`) and `filename` (the pathname component being resolved). The function essentially invokes `__d_lookup( )` first to search the `dentry` object of the component in the `dentry` cache. If no such object exists, `do_lookup( )` invokes `real_lookup( )`. This latter function reads the directory from disk by executing the `lookup` method of the `inode`, creates a new `dentry` object and inserts it in the `dentry` cache, then creates a new `inode` object and inserts it into the `inode` cache.^[*] At the end of this step, the `dentry` and `mnt` fields of the `next` local variable will point, respectively, to the `dentry` object and the mounted filesystem object of the component name to be resolved in this cycle.

[*] In a few cases, the function might find the required `inode` already in the `inode` cache. This happens when the pathname component is the last one and it does not refer to a directory, the corresponding file has several hard links, and finally the file has been recently accessed through a hard link different from the one used in this pathname.

- k. Invokes the `follow_mount( )` function to check whether the component just resolved (`next.dentry`) refers to a directory that is a mount point for some filesystem (`next.dentry->d_mounted` is greater than zero). `follow_mount( )` updates `next.dentry` and `next.mnt` so that they point to the `dentry` object and mounted filesystem object of the upmost filesystem mounted on the directory specified by this pathname component (see step 5g).
 - l. Checks whether the component just resolved refers to a symbolic link (`next.dentry->d_inode` has a custom `follow_link` method). We'll deal with this case in the later section "[Lookup of Symbolic Links](#)."
 - m. Checks whether the component just resolved refers to a directory (`next.dentry->d_inode` has a custom `lookup` method). If not, returns the error `-ENOTDIR`, because the component is in the middle of the original pathname.
 - n. Sets `nd->dentry` to `next.dentry` and `nd->mnt` to `next.mnt`, then continues with the next component of the pathname.
6. Now all components of the original pathname are resolved except the last one. Clears the `LOOKUP_CONTINUE` flag in `nd->flags`.
 7. If the pathname has a trailing slash, it sets the `LOOKUP_FOLLOW` and `LOOKUP_DIRECTORY` flags in the `lookup_flags` local variable to force the last component to be interpreted by later functions as a directory name.
 8. Checks the value of the `LOOKUP_PARENT` flag in the `lookup_flags` variable. In the following, we assume that the flag is set to 0, and we postpone the opposite case to the next section.
 9. If the name of the last component is "." (a single dot), terminates the execution and returns the value 0 (no error). In the `nameidata` structure that `nd` points to, the `dentry` and `mnt` fields refer to the objects relative to the next-to-last component of the pathname (each component "." has no effect inside a pathname).
 10. If the name of the last component is ".." (two dots), it tries to climb to the parent directory:
 - a. If the last resolved directory is the process's root directory (`nd->dentry` is equal to `current->fs->root` and `nd->mnt` is equal to `current->fs->rootmnt`), it invokes `follow_mount( )` on the next-to-last component and terminates the execution and returns the value 0 (no error). `nd->dentry` and `nd->mnt` refer to the objects relative to the next-to-last component of the pathname that is, to the root directory of the process.
 - b. If the last resolved directory is the root directory of the `nd->mnt` filesystem (`nd->dentry` is equal to `nd->mnt->mnt_root`) and the `nd->mnt` filesystem is not mounted on top of another filesystem (`nd->mnt` is equal to `nd->mnt->mnt_parent`), then climbing is impossible, thus invokes `follow_mount( )` on the next-to-last component and terminates the execution and returns the value 0 (no error).
 - c. If the last resolved directory is the root directory of the `nd->mnt` filesystem and the `nd->mnt` filesystem is mounted on top of another filesystem, it sets `nd->dentry` to `nd->mnt->mnt_mountpoint` and `nd->mnt` to `nd->mnt->mnt_parent`, then

restarts step 10.

- d. If the last resolved directory is not the root directory of a mounted filesystem, it sets `nd->dentry` to `nd->dentry->d_parent`, invokes `follow_mount ( )` on the parent directory, and terminates the execution and returns the value 0 (no error). `nd->dentry` and `nd->mnt` refer to the objects relative to the component preceding the next-to-last component of the pathname.
11. The name of the last component is neither "." nor "..", so the function must look it up in the dentry cache. If the low-level filesystem has a custom `d_hash` dentry method, the function invokes it to modify the hash value already computed in step 5c.
12. Invokes `do_lookup ( )` to derive the dentry object associated with the parent directory and the filename (see step 5j). At the end of this step, the `next` local variable contains the pointers to both the dentry and the mounted filesystem descriptor relative to the last component name.
13. Invokes `follow_mount ( )` to check whether the last component is a mount point for some filesystem and, if this is the case, to update the `next` local variable with the addresses of the dentry object and mounted filesystem object relative to the root directory of the upmost mounted filesystem.
14. Checks whether the `LOOKUP_FOLLOW` flag is set in `lookup_flags` and the inode object `next.dentry->d_inode` has a custom `follow_link` method. If this is the case, the component is a symbolic link that must be interpreted, as described in the later section "[Lookup of Symbolic Links](#)."
15. The component is not a symbolic link or the symbolic link should not be interpreted. Sets the `nd->mnt` and `nd->dentry` fields with the value stored in `next.mnt` and `next.dentry`, respectively. The final dentry object is the result of the whole lookup operation.
16. Checks whether `nd->dentry->d_inode` is `NULL`. This happens when there is no inode associated with the dentry object, usually because the pathname refers to a nonexistent file. In this case, the function returns the error code `-ENOENT`.
17. There is an inode associated with the last component of the pathname. If the `LOOKUP_DIRECTORY` flag is set in `lookup_flags`, it checks that the inode has a custom `lookup` method that is, it is a directory. If not, the function returns the error code `-ENOTDIR`.
18. Returns the value 0 (no error). `nd->dentry` and `nd->mnt` refer to the last component of the pathname.

12.5.2. Parent Pathname Lookup

In many cases, the real target of a lookup operation is not the last component of the pathname, but the next-to-last one. For example, when a file is created, the last component denotes the filename of the not yet existing file, and the rest of the pathname specifies the directory in which the new link must be inserted. Therefore, the lookup operation should fetch the dentry object of the next-to-last component. For another example, unlinking a file identified by the pathname `/foo/bar` consists of removing `bar` from the directory `foo`. Thus, the kernel is really interested in accessing the directory `foo` rather than `bar`.

The `LOOKUP_PARENT` flag is used whenever the lookup operation must resolve the directory containing the last component of the pathname, rather than the last component itself.

When the `LOOKUP_PARENT` flag is set, the `link_path_walk ( )` function also sets up the `last` and `last_type` fields of the `nameidata` data structure. The `last` field stores the name of the last component in the pathname. The `last_type` field identifies the type of the last component; it may be set to one of the values shown in [Table 12-17](#).

Table 12-17. The values of the `last_type` field in the `nameidata` data structure

Value	Description
<code>LAST_NORM</code>	Last component is a regular filename
<code>LAST_ROOT</code>	Last component is "/" (that is, the entire pathname is "/")
<code>LAST_DOT</code>	Last component is "."
<code>LAST_DOTDOT</code>	Last component is ".."
<code>LAST_BIND</code>	Last component is a symbolic link into a special filesystem

The `LAST_ROOT` flag is the default value set by `path_lookup( )` when the whole pathname lookup operation starts (see the description at the beginning of the section "[Pathname Lookup](#)"). If the pathname turns out to be simply "/", the kernel does not change the initial value of the `last_type` field.

The remaining values of the `last_type` field are set by `link_path_walk( )` when the `LOOKUP_PARENT` flag is set; in this case, the function performs the same steps described in the previous section up to step 8. From step 8 onward, however, the lookup operation for the last component of the pathname is different:

1. Sets `nd->last` to the name of the last component.
2. Initializes `nd->last_type` to `LAST_NORM`.
3. If the name of the last component is "." (a single dot), it sets `nd->last_type` to `LAST_DOT`.
4. If the name of the last component is ".." (two dots), it sets `nd->last_type` to `LAST_DOTDOT`.
5. Returns the value 0 (no error).

As you can see, the last component is not interpreted at all. Thus, when the function terminates, the `dentry` and `mnt` fields of the `nameidata` data structure point to the objects relative to the directory that includes the last component.

12.5.3. Lookup of Symbolic Links

Recall that a symbolic link is a regular file that stores a pathname of another file. A pathname may include symbolic links, and they must be resolved by the kernel.

For example, if `/foo/bar` is a symbolic link pointing to (containing the pathname) `../dir`, the pathname `/foo/bar/file` must be resolved by the kernel as a reference to the file `/dir/file`. In this example, the kernel must perform two different lookup operations. The first one resolves `/foo/bar`: when the kernel discovers that `bar` is the name of a symbolic link, it must retrieve its content and interpret it as another pathname. The second pathname operation starts from the directory reached by the first operation and continues until the last component of the symbolic link pathname has been resolved. Next, the original lookup operation resumes from the dentry reached in the second one and with the component following the symbolic link in the original pathname.

To further complicate the scenario, the pathname included in a symbolic link may include other symbolic links. You might think that the kernel code that resolves the symbolic links is hard to understand, but this is not true; the code is actually quite simple because it is recursive.

However, untamed recursion is intrinsically dangerous. For instance, suppose that a symbolic link points to itself. Of course, resolving a pathname including such a symbolic link may induce an endless stream of recursive invocations, which in turn quickly leads to a kernel stack overflow. The `link_count` field in the descriptor of the current process is used to avoid the problem: the field is increased before each recursive execution and decreased right after. If a sixth nested lookup operation is attempted, the whole lookup operation terminates with an error code. Therefore, the level of nesting of symbolic links can be at most 5.

Furthermore, the `total_link_count` field in the descriptor of the current process keeps track of how many symbolic links (even nonnested) were followed in the original lookup operation. If this counter reaches the value 40, the lookup operation aborts. Without this counter, a malicious user could create a pathological pathname including many consecutive symbolic links that freeze the kernel in a very long lookup operation.

This is how the code basically works: once the `link_path_walk( )` function retrieves the dentry object associated with a component of the pathname, it checks whether the corresponding inode object has a custom `follow_link` method (see step 51 and step 14 in the section "[Standard Pathname Lookup](#)"). If so, the inode is a symbolic link that must be interpreted before proceeding with the lookup operation of the original pathname.

In this case, the `link_path_walk( )` function invokes `do_follow_link( )`, passing to it the address dentry of the dentry object of the symbolic link and the address `nd` of the `nameidata` data structure. In turn, `do_follow_link( )` performs the following steps:

1. Checks that `current->link_count` is less than 5; otherwise, it returns the error code `-ELOOP`.
2. Checks that `current->total_link_count` is less than 40; otherwise, it returns the error code `-ELOOP`.
3. Invokes `cond_resched( )` to perform a process switch if required by the current process (flag `TIF_NEED_RESCHED` in the `thread_info` descriptor of the current process set).
4. Increases `current->link_count`, `current->total_link_count`, and `nd->depth`.
5. Updates the access time of the inode object associated with the symbolic link to be resolved.
6. Invokes the filesystem-dependent function that implements the `follow_link` method passing to it the dentry and `nd` parameters. This function extracts the pathname stored in the symbolic link's inode, and saves this pathname in the proper entry of the `nd->saved_names` array.
7. Invokes the `_vfs_follow_link( )` function passing to it the address `nd` and the address of the pathname in the `nd->saved_names` array (see below).
8. If defined, executes the `put_link` method of the inode object, thus releasing the temporary data structures allocated by the `follow_link` method.
9. Decreases the `current->link_count` and `nd->depth` fields.
10. Returns the error code returned by the `_vfs_follow_link( )` function (0 for no error).

In turn, the `_vfs_follow_link( )` does essentially the following:

1. Checks whether the first character of the pathname stored in the symbolic link is a slash: in this case an absolute pathname has been found, so there is no need to keep in memory any information about the previous path. If so, invokes `path_release( )` on the `nameidata` structure, thus releasing the objects resulting from the previous lookup steps; then, the function sets the `dentry` and `mnt` fields of the `nameidata` data structure to the current process root directory.
2. Invokes `link_path_walk( )` to resolve the symbolic link pathname, passing to it as parameters the pathname and `nd`.
3. Returns the value taken from `link_path_walk( )`.

When `do_follow_link( )` finally terminates, it has set the `dentry` field of the `next` local variable with the address of the `dentry` object referred to by the symbolic link to the original execution of `link_path_walk( )`. The `link_path_walk( )` function can then proceed with the next step.

12.6. Implementations of VFS System Calls

For the sake of brevity, we cannot discuss the implementation of all the VFS system calls listed in [Table 12-1](#). However, it could be useful to sketch out the implementation of a few system calls, in order to show how VFS's data structures interact.

Let's reconsider the example proposed at the beginning of this chapter: a user issues a shell command that copies the MS-DOS file */floppy/TEST* to the Ext2 file */tmp/test*. The command shell invokes an external program such as *cp*, which we assume executes the following code fragment:

```
inf = open("/floppy/TEST", O_RDONLY, 0);
outf = open("/tmp/test", O_WRONLY | O_CREAT | O_TRUNC, 0600);
do {
    len = read(inf, buf, 4096);
    write(outf, buf, len);
} while (len);
close(outf);
close(inf);
```

Actually, the code of the real *cp* program is more complicated, because it must also check for possible error codes returned by each system call. In our example, we focus our attention on the "normal" behavior of a copy operation.

12.6.1. The open() System Call

The `open( )` system call is serviced by the `sys_open( )` function, which receives as its parameters the pathname `filename` of the file to be opened, some access mode flags `flags`, and a permission bit mask `mode` if the file must be created. If the system call succeeds, it returns a file descriptor that is, the index assigned to the new file in the `current->files->fd` array of pointers to file objects; otherwise, it returns `-1`.

In our example, `open( )` is invoked twice; the first time to open */floppy/TEST* for reading (`O_RDONLY` flag) and the second time to open */tmp/test* for writing (`O_WRONLY` flag). If */tmp/test* does not already exist, it is created (`O_CREAT` flag) with exclusive read and write access for the owner (octal `0600` number in the third parameter).

Conversely, if the file already exists, it is rewritten from scratch (`O_TRUNC` flag). [Table 12-18](#) lists all flags of the `open( )` system call.

Table 12-18. The flags of the open() system call

Flag name	Description
<code>O_RDONLY</code>	Open for reading
<code>O_WRONLY</code>	Open for writing
<code>O_RDWR</code>	Open for both reading and writing

O_CREAT	Create the file if it does not exist
O_EXCL	With O_CREAT, fail if the file already exists
O_NOCTTY	Never consider the file as a controlling terminal
O_TRUNC	Truncate the file (remove all existing contents)
O_APPEND	Always write at end of the file
O_NONBLOCK	No system calls will block on the file
O_NDELAY	Same as O_NONBLOCK
O_SYNC	Synchronous write (block until physical write terminates)
FASYNC	I/O event notification via signals
O_DIRECT	Direct I/O transfer (no kernel buffering)
O_LARGEFILE	Large file (size greater than 2 GB)
O_DIRECTORY	Fail if file is not a directory
O_NOFOLLOW	Do not follow a trailing symbolic link in pathname
O_NOATIME	Do not update the inode's last access time

Let's describe the operation of the `sys_open ( )` function. It performs the following steps:

1. Invokes `getname ( )` to read the file pathname from the process address space.
2. Invokes `get_unused_fd ( )` to find an empty slot in `current->files->fd`. The corresponding index (the new file descriptor) is stored in the `fd` local variable.
3. Invokes the `filp_open ( )` function, passing as parameters the pathname, the access mode flags, and the permission bit mask. This function, in turn, executes the following steps:
 - a. Copies the access mode flags into `namei__flags`, but encodes the access mode flags `O_RDONLY`, `O_WRONLY`, and `O_RDWR` with a special format: the bit at index 0 (lowest-order) of `namei__flags` is set only if the file access requires read privileges;

similarly, the bit at index 1 is set only if the file access requires write privileges. Notice that it is not possible to specify in the `open ( )` system call that a file access does not require either read or write privileges; this makes sense, however, in a pathname lookup operation involving symbolic links.

- b. Invokes `open_namei ( )`, passing to it the pathname, the modified access mode flags, and the address of a local `nameidata` data structure. The function performs the lookup operation in the following manner:

- ◊ If `O_CREAT` is not set in the access mode flags, starts the lookup operation with the `LOOKUP_PARENT` flag not set and the `LOOKUP_OPEN` flag set. Moreover, the `LOOKUP_FOLLOW` flag is set only if `O_NOFOLLOW` is cleared, while the `LOOKUP_DIRECTORY` flag is set only if the `O_DIRECTORY` flag is set.
- ◊ If `O_CREAT` is set in the access mode flags, starts the lookup operation with the `LOOKUP_PARENT`, `LOOKUP_OPEN`, and `LOOKUP_CREATE` flags set. Once the `path_lookup ( )` function successfully returns, checks whether the requested file already exists. If not, allocates a new disk inode by invoking the `create` method of the parent inode.

The `open_namei ( )` function also executes several security checks on the file located by the lookup operation. For instance, the function checks whether the inode associated with the dentry object found really exists, whether it is a regular file, and whether the current process is allowed to access it according to the access mode flags. Also, if the file is opened for writing, the function checks that the file is not locked by other processes.

- c. Invokes the `dentry_open ( )` function, passing to it the addresses of the dentry object and the mounted filesystem object located by the lookup operation, and the access mode flags. In turn, this function:

1. Allocates a new file object.
2. Initializes the `f_flags` and `f_mode` fields of the file object according to the access mode flags passed to the `open ( )` system call.
3. Initializes the `f_dentry` and `f_vfsmnt` fields of the file object according to the addresses of the dentry object and the mounted filesystem object passed as parameters.
4. Sets the `f_op` field to the contents of the `i_fop` field of the corresponding inode object. This sets up all the methods for future file operations.
5. Inserts the file object into the list of opened files pointed to by the `s_files` field of the filesystem's superblock.
6. If the `open` method of the file operations is defined, the function invokes it.
7. Invokes `file_ra_state_init ( )` to initialize the read-ahead data structures (see [Chapter 16](#)).
8. If the `O_DIRECT` flag is set, it checks whether direct I/O operations can be performed on the file (see [Chapter 16](#)).
9. Returns the address of the file object.

- d. Returns the address of the file object.

- 4. Sets `current->files->fd[fd]` to the address of the file object returned by `dentry_open ( )`.

- 5. Returns `fd`.

12.6.2. The `read ( )` and `write ( )` System Calls

Let's return to the code in our `cp` example. The `open ( )` system calls return two file descriptors, which are stored in the `inf` and `outf` variables. Then the program starts a loop: at each iteration, a portion of the

/floppy/TEST file is copied into a local buffer (`read( )` system call), and then the data in the local buffer is written into the */tmp/test* file (`write( )` system call).

The `read( )` and `write( )` system calls are quite similar. Both require three parameters: a file descriptor `fd`, the address `buf` of a memory area (the buffer containing the data to be transferred), and a number `count` that specifies how many bytes should be transferred. Of course, `read( )` transfers the data from the file into the buffer, while `write( )` does the opposite. Both system calls return either the number of bytes that were successfully transferred or `-1` to signal an error condition.

A return value less than `count` does not mean that an error occurred. The kernel is always allowed to terminate the system call even if not all requested bytes were transferred, and the user application must accordingly check the return value and reissue, if necessary, the system call. Typically, a small value is returned when reading from a pipe or a terminal device, when reading past the end of the file, or when the system call is interrupted by a signal. The end-of-file condition (EOF) can easily be recognized by a zero return value from `read( )`. This condition will not be confused with an abnormal termination due to a signal, because if `read( )` is interrupted by a signal before a data is read, an error occurs.

The read or write operation always takes place at the file offset specified by the current file pointer (field `f_pos` of the file object). Both system calls update the file pointer by adding the number of transferred bytes to it.

In short, both `sys_read( )` (the `read( )`'s service routine) and `sys_write( )` (the `write( )`'s service routine) perform almost the same steps:

1. Invokes `fget_light( )` to derive from `fd` the address `file` of the corresponding file object (see the earlier section ["Files Associated with a Process"](#)).
2. If the flags in `file->f_mode` do not allow the requested access (read or write operation), it returns the error code `-EBADF`.
3. If the file object does not have a `read( )` or `aio_read( )` (`write( )` or `aio_write( )`) file operation, it returns the error code `-EINVAL`.
4. Invokes `access_ok( )` to perform a coarse check on the `buf` and `count` parameters (see the section ["Verifying the Parameters"](#) in [Chapter 10](#)).
5. Invokes `rw_verify_area( )` to check whether there are conflicting mandatory locks for the file portion to be accessed. If so, it returns an error code, or puts the current process to sleep if the lock has been requested with a `F_SETLK` command (see the section ["File Locking"](#) later in this chapter).
6. If defined, it invokes either the `file->f_op->read` or `file->f_op->write` method to transfer the data; otherwise, invokes either the `file->f_op->aio_read` or `file->f_op->aio_write` method. All these methods, which are discussed in [Chapter 16](#), return the number of bytes that were actually transferred. As a side effect, the file pointer is properly updated.
7. Invokes `fput_light( )` to release the file object.
8. Returns the number of bytes actually transferred.

12.6.3. The `close( )` System Call

The loop in our example code terminates when the `read( )` system call returns the value 0 that is, when all bytes of */floppy/TEST* have been copied into */tmp/test*. The program can then close the open files, because the copy operation has completed.

The `close( )` system call receives as its parameter `fd`, which is the file descriptor of the file to be closed. The `sys_close( )` service routine performs the following operations:

1. Gets the file object address stored in `current->files->fd[fd]`; if it is `NULL`, returns an error code.
2. Sets `current->files->fd[fd]` to `NULL`. Releases the file descriptor `fd` by clearing the corresponding bits in the `open_fds` and `close_on_exec` fields of `current->files` (see [Chapter 20](#) for the Close on Execution flag).
3. Invokes `filp_close( )`, which performs the following operations:
 - a. Invokes the `flush` method of the file operations, if defined.
 - b. Releases all mandatory locks on the file, if any (see next section).
 - c. Invokes `fput( )` to release the file object.
4. Returns 0 or an error code. An error code can be raised by the `flush` method or by an error in a previous write operation on the file.

12.7. File Locking

When a file can be accessed by more than one process, a synchronization problem occurs. What happens if two processes try to write in the same file location? Or again, what happens if a process reads from a file location while another process is writing into it?

In traditional Unix systems, concurrent accesses to the same file location produce unpredictable results. However, Unix systems provide a mechanism that allows the processes to lock a file region so that concurrent accesses may be easily avoided.

The POSIX standard requires a file-locking mechanism based on the `fcntl( )` system call. It is possible to lock an arbitrary region of a file (even a single byte) or to lock the whole file (including data appended in the future). Because a process can choose to lock only a part of a file, it can also hold multiple locks on different parts of the file.

This kind of lock does not keep out another process that is ignorant of locking. Like a semaphore used to protect a critical region in code, the lock is considered "advisory" because it doesn't work unless other processes cooperate in checking the existence of a lock before accessing the file. Therefore, POSIX's locks are known as advisory locks .

Traditional BSD variants implement advisory locking through the `flock( )` system call. This call does not allow a process to lock a file region, only the whole file. Traditional System V variants provide the `lockf( )` library function, which is simply an interface to `fcntl( )` .

More importantly, System V Release 3 introduced mandatory locking: the kernel checks that every invocation of the `open( )` , `read( )` , and `write( )` system calls does not violate a mandatory lock on the file being accessed. Therefore, mandatory locks are enforced even between noncooperative processes.^[*]

[*] Oddly enough, a process may still unlink (delete) a file even if some other process owns a mandatory lock on it! This perplexing situation is possible because when a process deletes a file hard link, it does not modify its contents, but only the contents of its parent directory.

Whether processes use advisory or mandatory locks, they can use both shared read locks and exclusive write locks . Several processes may have read locks on some file region, but only one process can have a write lock on it at the same time. Moreover, it is not possible to get a write lock when another process owns a read lock for the same file region, and vice versa.

12.7.1. Linux File Locking

Linux supports all types of file locking: advisory and mandatory locks, plus the `fcntl( )` and `flock( )` system calls (`lockf( )` is implemented as a standard library function).

The expected behavior of the `flock( )` system call in every Unix-like operating system is to produce advisory locks only, without regard for the `MS_MANDLOCK` mount flag. In Linux, however, a special kind of `flock( )`'s mandatory lock is used to support some proprietary network filesystems. It is the so-called share-mode mandatory lock; when set, no other process may open a file that would conflict with the access mode of the lock. Use of this feature for native Unix applications is discouraged, because the resulting source code will be nonportable.

Another kind of `fcntl( )`-based mandatory lock called lease has been introduced in Linux. When a process tries to open a file protected by a lease, it is blocked as usual. However, the process that owns the lock receives a signal. Once informed, it should first update the file so that its content is consistent, and then release the lock. If the owner does not do this in a well-defined time interval (tunable by writing a number of seconds into `/proc/sys/fs/lease-break-time`, usually 45 seconds), the lease is automatically removed by the kernel and the blocked process is allowed to continue.

A process can get or release an advisory file lock on a file in two possible ways:

- By issuing the `flock( )` system call. The two parameters of the system call are the `fd` file descriptor, and a command to specify the lock operation. The lock applies to the whole file.
- By using the `fcntl( )` system call. The three parameters of the system call are the `fd` file descriptor, a command to specify the lock operation, and a pointer to a `flock` structure (see [Table 12-20](#)). A couple of fields in this structure allow the process to specify the portion of the file to be locked. Processes can thus hold several locks on different portions of the same file.

Both the `fcntl( )` and the `flock( )` system call may be used on the same file at the same time, but a file locked through `fcntl( )` does not appear locked to `flock( )`, and vice versa. This has been done on purpose in order to avoid the deadlocks occurring when an application using a type of lock relies on a library that uses the other type.

Handling mandatory file locks is a bit more complex. Here are the steps to follow:

1. Mount the filesystem where mandatory locking is required using the `-o mand` option in the `mount` command, which sets the `MS_MANDLOCK` flag in the `mount( )` system call. The default is to disable mandatory locking.
2. Mark the files as candidates for mandatory locking by setting

their set-group bit (SGID) and clearing the group-execute permission bit. Because the set-group bit makes no sense when the group-execute bit is off, the kernel interprets that combination as a hint to use mandatory locks instead of advisory ones.

3. Uses the `fcntl( )` system call (see below) to get or release a file lock.

Handling leases is much simpler than handling mandatory locks: it is sufficient to invoke a `fcntl( )` system call with a `F_SETLEASE` or `F_GETLEASE` command. Another `fcntl( )` invocation with the `F_SETSIG` command may be used to change the type of signal to be sent to the lease process holder.

Besides the checks in the `read( )` and `write( )` system calls, the kernel takes into consideration the existence of mandatory locks when servicing all system calls that could modify the contents of a file. For instance, an `open( )` system call with the `O_TRUNC` flag set fails if any mandatory lock exists for the file.

The following section describes the main data structure used by the kernel to handle file locks issued by means of the `flock( )` system call (`FL_FLOCK` locks) and of the `fcntl( )` system call (`FL_POSIX` locks).

12.7.2. File-Locking Data Structures

All type of Linux locks are represented by the same `file_lock` data structure whose fields are shown in [Table 12-19](#).

Table 12-19. The fields of the `file_lock` data structure

Type	Field	Description
<code>struct file_lock *</code>	<code>fl_next</code>	Next element in list of locks associated with the inode
<code>struct list_head</code>	<code>fl_link</code>	Pointers for active or blocked list
<code>struct list_head</code>	<code>fl_block</code>	Pointers for the lock's waiters list
<code>struct files_struct *</code>	<code>fl_owner</code>	Owner's <code>files_struct</code>
<code>unsigned int</code>	<code>fl_pid</code>	PID of the process owner
<code>wait_queue_head_t</code>	<code>fl_wait</code>	Wait queue of blocked processes
<code>struct file *</code>	<code>fl_file</code>	Pointer to file object
<code>unsigned char</code>	<code>fl_flags</code>	Lock flags
<code>unsigned char</code>	<code>fl_type</code>	Lock type
<code>loff_t</code>	<code>fl_start</code>	Starting offset of locked region
<code>loff_t</code>	<code>fl_end</code>	

		Ending offset of locked region
<code>struct fasync_struct *</code>	<code>fl_fasync</code>	Used for lease break notifications
<code>unsigned long</code>	<code>fl_break_time</code>	Remaining time before end of lease
<code>struct file_lock_operations *</code>	<code>fl_ops</code>	Pointer to file lock operations
<code>struct lock_manager_operations *</code>	<code>fl_mops</code>	Pointer to lock manager operations
<code>union</code>	<code>fl_u</code>	Filesystem-specific information

All `lock_file` structures that refer to the same file on disk are collected in a singly linked list, whose first element is pointed to by the `i_flock` field of the inode object. The `fl_next` field of the `lock_file` structure specifies the next element in the list.

When a process issues a blocking system call to require an exclusive lock while there are shared locks on the same file, the lock request cannot be satisfied immediately and the process must be suspended. The process is thus inserted into a wait queue pointed to by the `fl_wait` field of the blocked lock's `file_lock` structure. Two lists are used to distinguish lock requests that have been satisfied (active locks) from those that cannot be satisfied right away (blocked locks).

All active locks are linked together in the "global file lock list" whose head element is stored in the `file_lock_list` variable. Similarly, all blocked locks are linked together in the "blocked list" whose head element is stored in the `blocked_list` variable. The `fl_link` field is used to insert a `lock_file` structure in either one of these two lists.

Last but not least, the kernel must keep track of all blocked locks (the "waiters") associated with a given active lock (the "blocker"): this is the purpose of a list that links together all waiters with respect to a given blocker. The `fl_block` field of the blocker is the dummy head of the list, while the `fl_block` fields of the waiters store the pointers to the adjacent elements in the list.

12.7.3. FL_FLOCK Locks

An `FL_FLOCK` lock is always associated with a file object and is thus owned by the process that opened the file (or by all clone processes sharing the same opened file). When a lock is requested and granted, the kernel replaces every other lock that the process is holding on the same file object with the new lock. This happens only when a process wants to change an already owned read lock into a write one, or vice versa. Moreover, when a file object is being freed by the `fput ( )` function, all `FL_FLOCK` locks that refer to the file object are destroyed. However, there could be other `FL_FLOCK` read locks set by other processes for the same file (inode), and they still remain active.

The `flock ( )` system call allows a process to apply or remove an advisory lock on an open file. It acts on two parameters: the `fd` file descriptor of the file to be acted upon and a `cmd` parameter that specifies the lock operation. A `cmd` parameter of `LOCK_SH` requires a shared lock for reading, `LOCK_EX` requires an exclusive lock for writing, and `LOCK_UN` releases the lock.^[*]

^[*] Actually, the `flock ( )` system call can also establish share-mode mandatory locks by specifying the command `LOCK_MAND`. However, we'll not further discuss this case.

Usually this system call blocks the current process if the request cannot be immediately satisfied, for instance if the process requires an exclusive lock while some other process has already acquired the same lock. However, if the `LOCK_NB` flag is passed together with the `LOCK_SH` or `LOCK_EX` operation, the system call does not block; in other words, if the lock cannot be immediately obtained, the system call returns an error code.

When the `sys_flock ( )` service routine is invoked, it performs the following steps:

1. Checks whether `fd` is a valid file descriptor; if not, returns an error code. Gets the address `filp` of the corresponding file object.
2. Checks that the process has read and/or write permission on the open file; if not, returns an error code.
3. Gets a new `file_lock` object `lock` and initializes it in the appropriate way: the `fl_type` field is set according to the value of the parameter `cmd`, the `fl_file` field is set to the address `filp` of the file object, the `fl_flags` field is set to `FL_FLOCK`, the `fl_pid` field is set to `current->tgid`, and the `fl_end` field is set to `-1` to denote the fact that locking refers to the whole file (and not to a portion of it).
4. If the `cmd` parameter does not include the `LOCK_NB` bit, it adds to the `fl_flags` field the `FL_SLEEP` flag.
5. If the file has a `flock` file operation, the routine invokes it, passing as its parameters the file object pointer `filp`, a flag (`F_SETLKW` or `F_SETLK` depending on the value of the `LOCK_NB` bit), and the address of the new `file_lock` object `lock`.
6. Otherwise, if the `flock` file operation is not defined (the common case), invokes `flock_lock_file_wait ( )` to try to perform the required lock operation. Two parameters are passed: `filp`, a file object pointer, and `lock`, the address of the new `file_lock` object created in step 3.
7. If the `file_lock` descriptor has not been inserted in the active or blocked lists in the previous step, the routine releases it.
8. Returns 0 in case of success.

The `flock_lock_file_wait ( )` function executes a cycle consisting of the following steps:

1. Invokes `flock_lock_file ( )` passing as parameters the file object pointer `filp` and the address of the new `file_lock` object `lock`. This function performs, in turn, the following operations:
 - a. Searches the list that `filp->f_dentry->d_inode->i_flock` points to. If an `FL_FLOCK` lock for the same file object is found, checks its type (`LOCK_SH` or `LOCK_EX`): if it is equal to the type of the new lock, returns 0 (nothing has to be done). Otherwise, the function removes the old element from the list of locks on the inode and the global file lock list, wakes up all processes sleeping in the wait queues of the locks in the `fl_block` list, and frees the `file_lock` structure.
 - b. If the process is performing an unlock (`LOCK_UN`), nothing else needs to be done: the lock was nonexistent or it has already been released, thus returns 0.
 - c. If an `FL_FLOCK` lock for the same file object has been found thus the process is changing an already owned read lock into a write one (or vice versa) gives some other higher-priority process, in particular every process previously blocked on the old file lock, a chance to run by invoking `cond_resched ( )`.
 - d. Searches the list of locks on the inode again to verify that no existing `FL_FLOCK` lock conflicts with the requested one. There must be no `FL_FLOCK` write lock in the list, and moreover, there must be no `FL_FLOCK` lock at all if the process is requesting a write lock.
 - e. If no conflicting lock exists, it inserts the new `file_lock` structure into the inode's lock list and into the global file lock list, then returns 0 (success).
 - f. A conflicting lock has been found: if the `FL_SLEEP` flag in the `fl_flags` field is set, it inserts the new lock (the waiter lock) in the circular list of the blocker lock and in the global

- blocked list.
- g. Returns the error code `-EAGAIN`.
- 2. Checks the return code of `flock_lock_file( )`:
 - a. If the return code is 0 (no conflicting locks), it returns 0 (success).
 - b. There are incompatibilities. If the `FL_SLEEP` flag in the `fl_flags` field is cleared, it releases the `lock_file_lock` descriptor and returns `-EAGAIN`.
 - c. Otherwise, there are incompatibilities but the process can sleep: invokes `wait_event_interruptible( )` to insert the current process in the `lock->fl_wait` wait queue and to suspend it. When the process is awakened (right after the blocker lock has been released), it jumps to step 1 to retry the operation.

12.7.4. FL_POSIX Locks

An `FL_POSIX` lock is always associated with a process and with an inode; the lock is automatically released either when the process dies or when a file descriptor is closed (even if the process opened the same file twice or duplicated a file descriptor). Moreover, `FL_POSIX` locks are never inherited by a child across a `fork( )`.

When used to lock files, the `fcntl( )` system call acts on three parameters: the `fd` file descriptor of the file to be acted upon, a `cmd` parameter that specifies the lock operation, and an `fl` pointer to a `flock` data structure^[*] stored in the User Mode process address space; its fields are described in [Table 12-20](#).

[*] Linux also defines a `flock64` structure, which uses 64-bit long integers for the `offset` and `length` fields. In the following, we focus on the `flock` data structure, but the description is valid for `flock64` too.

Table 12-20. The fields of the flock data structure

Type	
Field	
Description	
short	
<code>l_type</code>	
	<code>F_RDLCK</code> (requests a shared lock), <code>F_WRLCK</code> (requests an exclusive lock), <code>F_UNLCK</code> (releases the lock)
short	
<code>l_whence</code>	
	<code>SEEK_SET</code> (from beginning of file), <code>SEEK_CURRENT</code> (from current file pointer), <code>SEEK_END</code> (from end of file)
<code>off_t</code>	
<code>l_start</code>	

Initial offset of the locked region relative to the value of `l_whence`

`off_t`

`l_len`

Length of locked region (0 means that the region includes all potential writes past the current end of the file)

`pid_t`

`l_pid`

PID of the owner

The `sys_fcntl( )` service routine behaves differently, depending on the value of the flag set in the `cmd` parameter:

`F_GETLK`

Determines whether the lock described by the `flock` structure conflicts with some `FL_POSIX` lock already obtained by another process. In this case, the `flock` structure is overwritten with the information about the existing lock.

`F_SETLK`

Sets the lock described by the `flock` structure. If the lock cannot be acquired, the system call returns an error code.

`F_SETLKW`

Sets the lock described by the `flock` structure. If the lock cannot be acquired, the system call blocks; that is, the calling process is put to sleep until the lock is available.

`F_GETLK64`, `F_SETLK64`, `F_SETLKW64`

Identical to the previous ones, but the `flock64` data structure is used rather than `flock`.

The `sys_fcntl( )` service routine gets first a file object corresponding to the `fd` parameter and invokes then `fcntl_getlk( )` or `fcntl_setlk( )`, depending on the command passed as its parameter (`F_GETBLK` for the former function, `F_SETLK` or `F_SETLKW` for the latter one). We'll consider the second case only.

The `fcntl_setlk( )` function acts on three parameters: a `filp` pointer to the file object, a `cmd` command (`F_SETLK` or `F_SETLKW`), and a pointer to a `flock` data structure. The steps performed are the following:

1. Reads the structure pointed to by the `fl` parameter in a local variable of type `flock`.
2. Checks whether the lock should be a mandatory one and the file has a shared memory mapping (see the section "[Memory Mapping](#)" in [Chapter 16](#)). In this case, the function refuses to create the lock and returns the `-EAGAIN` error code, because the file is already being accessed by another process.
3. Initializes a new `file_lock` structure according to the contents of the user's `flock` structure and to

the file size stored in the file's inode.

4. If the command is `F_SETLK`, the function sets the `FL_SLEEP` flag in the `fl_flags` field of the `file_lock` structure.
5. If the `l_type` field in the `flock` structure is equal to `F_RDLCK`, it checks whether the process is allowed to read from the file; similarly, if `l_type` is equal to `F_WRLCK`, checks whether the process is allowed to write into the file. If not, it returns an error code.
6. Invokes the `lock` method of the file operations, if defined. Usually for disk-based filesystems, this method is not defined.
7. Invokes `__posix_lock_file()` passing as parameters the address of the file's inode object and the address of the `file_lock` object. This function performs, in turn, the following operations:
 - a. Invokes `posix_locks_conflict()` for each `FL_POSIX` lock in the inode's lock list. The function checks whether the lock conflicts with the requested one. Essentially, there must be no `FL_POSIX` write lock for the same region in the inode list, and there may be no `FL_POSIX` lock at all for the same region if the process is requesting a write lock. However, locks owned by the same process never conflict; this allows a process to change the characteristics of a lock it already owns.
 - b. If a conflicting lock is found, the function checks whether `fcntl()` was invoked with the `F_SETLK` command. If so, the current process must be suspended: invokes `posix_locks_deadlock()` to check that no deadlock condition is being created among processes waiting for `FL_POSIX` locks, then inserts the new lock (waiter lock) both in the blocker list of the conflicting lock (blocker lock) and in the blocked list, and finally returns an error code. Otherwise, if `fcntl()` was invoked with the `F_SETLK` command, returns an error code.
 - c. As soon as the inode's lock list includes no conflicting lock, the function checks all the `FL_POSIX` locks of the current process that overlap the file region that the current process wants to lock, and combines and splits adjacent areas as required. For example, if the process requested a write lock for a file region that falls inside a read-locked wider region, the previous read lock is split into two parts covering the nonoverlapping areas, while the central region is protected by the new write lock. In case of overlaps, newer locks always replace older ones.
 - d. Inserts the new `file_lock` structure in the global file lock list and in the inode list.
 - e. Returns the value 0 (success).
8. Checks the return code of `__posix_lock_file()`:
 - a. If the return code is 0 (no conflicting locks), it returns 0 (success).
 - b. There are incompatibilities. If the `FL_SLEEP` flag in the `fl_flags` field is cleared, it releases the new `file_lock` descriptor and returns `-EAGAIN`.
 - c. Otherwise, if there are incompatibilities but the process can sleep, it invokes `wait_event_interruptible()` to insert the current process in the `lock->fl_wait` wait queue and to suspend it. When the process is awakened (right after the blocker lock has been released), it jumps to step 7 to retry the operation.